



**DEVELOPMENT OF EDGE SENTINEL: A RESILIENT EDGE-BASED
INTELLIGENT VISUAL SURVEILLANCE SYSTEM FOR INFRASTRUCTURE-
CONSTRAINED ENVIRONMENTS**

BY

AUSTINE CHUKWUEBUKA RAPHAEL

SEN/20/5090

A PROJECT SUBMITTED TO

THE DEPARTMENT OF SOFTWARE ENGINEERING,

SCHOOL OF COMPUTING,

THE FEDERAL UNIVERSITY OF TECHNOLOGY, AKURE, ONDO STATE,

NIGERIA,

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF

BACHELOR OF TECHNOLOGY (B.TECH.) DEGREE IN SOFTWARE

ENGINEERING.

JULY, 2026

DECLARATION

I hereby declare that this project titled **“Development of Edge Sentinel: A Resilient Edge-Based Intelligent Visual Surveillance System for Infrastructure-Constrained Environments”** was carried out by me in the Department of Software Engineering, School of Computing, The Federal University of Technology, Akure, Ondo State, Nigeria.

I also declare that this project has not been submitted, either in part or in full, for the award of any degree or diploma in this or any other institution.

Candidates Name:

Austine Chukwuebuka Raphael

Signature: _____

Date: _____

CERTIFICATION

This is to certify that this project titled “**Development of Edge Sentinel: A Resilient Edge-Based Intelligent Visual Surveillance System for Infrastructure-Constrained Environments**” is the outcome of original research and development work carried out by **Austine Chukwuebuka Raphael** with matriculation number **SEN/20/5090** in the Department of Software Engineering, School of Computing, The Federal University of Technology, Akure, Ondo State, Nigeria.

The project has been read and approved as meeting the requirements for the award of Bachelor of Technology (B.Tech.) degree in Software Engineering.

Project Supervisor

Prof. K. G. Akintola

Signature: _____

Date: _____

Head of Department

Prof. K. G. Akintola

Signature: _____

Date: _____

DEDICATION

This project is dedicated to God Almighty, whose grace, strength, and guidance sustained me throughout the course of this work.

It is also dedicated to my parents, whose love, prayers, and support have been a constant source of encouragement.

ACKNOWLEDGEMENT

I thank God Almighty for His grace, strength, wisdom, and guidance throughout the course of this project. His help made it possible for me to complete this work.

I sincerely appreciate my project supervisor, Prof. K. G. Akintola, for his guidance, corrections, patience, and professional support during the development of this project. His supervision helped shape the direction and quality of the work.

I also appreciate the Head of Department and all the academic and non-academic staff of the Department of Software Engineering, School of Computing, The Federal University of Technology, Akure, for the knowledge, training, and support given throughout my course of study.

My heartfelt appreciation goes to my parents for their love, prayers, sacrifices, and constant encouragement. Their support has been one of my greatest sources of strength.

I also thank my colleagues and friends for their encouragement, discussions, and support during the course of this project. Their contributions, directly and indirectly, helped me stay motivated and focused.

Finally, I appreciate everyone who contributed in one way or another to the successful completion of this project.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CCTV	Closed-Circuit Television
CSI	Camera Serial Interface
FPS	Frames Per Second
GSM	Global System for Mobile Communications
HTTP	Hypertext Transfer Protocol
IR	Infrared
NoIR	No Infrared Filter
PIR	Passive Infrared
SMS	Short Message Service
UI	User Interface
YOLO	You Only Look Once

ABSTRACT

Surveillance systems are widely used for security monitoring, but many conventional systems remain passive because they depend on continuous human observation or later review of recorded footage. In infrastructure-constrained environments, this limitation is more serious because stable internet connectivity, reliable power supply, continuous monitoring, and advanced technical support cannot always be assumed. This project developed and evaluated Edge Sentinel, a resilient edge-based intelligent visual surveillance prototype designed to support local security monitoring under such constraints.

The system was implemented on Raspberry Pi-class hardware using local person detection, anonymous multi-object tracking, configurable behavioural indicators, event logging, evidence handling, persistent alert queueing, and GSM/SMS capability. EfficientDet-Lite0 was selected as the reference detector, while ByteTrack was used to maintain anonymous track continuity across frames. The implemented behavioural indicators covered restricted presence, lingering, crowd density, tailgating, and abnormal motion. The system was designed to avoid routine identity recognition and instead generate explainable security events based on zones, duration, count, transition, and movement conditions.

The prototype was evaluated through hardware verification, detector-tracker screening, behavioural scenario testing, sustained Raspberry Pi profiling, and alert queue recovery testing. The IMX219 NoIR camera, HC-SR501 PIR sensor, and SIMCOM A7670G modem were functionally verified. In the behavioural scenario evaluation, nine scenarios were tested, with eight correct outcomes, one false negative, and no false-positive scenarios. This produced a scenario-level precision of 1.000, recall of 0.875, and F1 score of approximately 0.933. Sustained saved-video profiling showed approximately 22.72 processed frames per second, no throttling, no undervoltage, no dropped or failed frames, and clean shutdown. During a simulated HTTP endpoint outage, ten alerts were queued and all ten were delivered after recovery, with no loss or duplication.

The results show that Edge Sentinel achieved its aim within the implemented scope. The project demonstrates that local person detection, anonymous tracking, explainable rule-based event reasoning, and persistent alert recovery can be combined into a modular edge surveillance architecture for infrastructure-constrained environments. The system does not claim perfect anomaly detection, formal detector accuracy, measured power consumption, complete low-light performance, or guaranteed communication reliability. Its contribution lies in showing how resilient local intelligence and alert preservation can strengthen surveillance systems where continuous cloud connectivity and stable infrastructure cannot be guaranteed.

Keywords: Edge surveillance, intelligent visual surveillance, edge AI, Raspberry Pi, behavioural event reasoning, alert resilience, infrastructure-constrained environments.

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Surveillance systems are used to support security monitoring, access control, incident review, and situational awareness in physical environments such as schools, hostels, laboratories, offices, farms, storage facilities, construction sites, and other sensitive locations. In many of these places, the camera is expected to act as an extra layer of awareness. However, traditional closed-circuit television systems mainly capture and store video footage for live viewing or later review. This makes them useful for evidence collection, but also largely passive, because their effectiveness still depends on a human operator watching the footage or checking the recording after an incident has already occurred.

This passive nature becomes a serious limitation when the monitored environment is busy, remote, poorly staffed, or difficult to supervise continuously. Human operators may miss important events when they are tired, distracted, or required to monitor many feeds at once. Ali et al. (2024), Duong et al. (2023), and Elmetwally et al. (2025) observed that manual video monitoring is time-consuming and prone to human error, especially where large amounts of surveillance footage must be reviewed. In the Nigerian context, Nte et al. (2020) also identified practical limitations affecting CCTV usefulness, including poor monitoring practices, lack of technical support, malfunctioning equipment, and limited operational effectiveness of installed systems.

Recent developments in computer vision and edge computing have made it possible for surveillance systems to become more active. Instead of only recording footage, an intelligent surveillance system can process visual input, detect persons or objects of interest, and generate alerts when defined security conditions occur. Such systems can help reduce dependence on continuous human monitoring by drawing attention to relevant events. This is important because a security system should not only preserve evidence after an incident. It should also support timely awareness while the incident is happening.

Despite this progress, many intelligent surveillance systems are still designed with assumptions that may not hold in infrastructure-constrained environments. Some systems depend on stable power supply, high-speed internet connectivity, cloud processing, or continuous video streaming. These conditions are not always available in Nigerian and similar deployment contexts, especially in rural facilities, semi-urban campuses, hostels, farms, construction sites, laboratories, and small warehouses. Nte et al. (2020) reported that

erratic power supply, voltage fluctuation, and lack of power can affect CCTV operation and even make installed systems redundant. Aliyu and Airoboman (2025) also noted that advanced surveillance solutions for Nigerian rural communities are often limited by internet dependence, power requirements, and affordability.

The challenge is therefore not only to make surveillance intelligent, but to make it resilient. A surveillance system that depends entirely on cloud connectivity may fail to deliver alerts when the network is weak. A system that requires heavy computation may not run reliably on low-cost edge hardware. A system that only records video may not provide timely awareness. For infrastructure-constrained environments, a useful surveillance system must be able to perform important functions locally, preserve event records, and continue operating in a limited form when remote communication is unavailable.

Another important issue is the computational cost of deep learning-based surveillance. Many modern computer vision and video anomaly detection methods achieve strong results, but they often require high processing power, large memory, graphics processing units, or stable energy supply. Cheng et al. (2018) described deep neural networks as resource-intensive, while Duong et al. (2023) noted that some temporal video analysis methods can be computationally demanding and unsuitable for energy-constrained systems. Low-cost edge devices such as Raspberry Pi boards have limited processing capacity, memory, storage, and power margin. This makes it necessary to balance detection performance with speed, simplicity, and deployment cost.

Lighting conditions also affect surveillance performance. Many security events occur in the evening, at night, or in poorly illuminated areas. Under low-light conditions, video frames may suffer from noise, low contrast, blur, weak edge information, and loss of detail. These degradations can reduce both human visibility and machine detection accuracy. Studies on low-light image enhancement and low-light detection show that poor illumination remains a major challenge for visual surveillance and object detection systems (Li et al., 2021; Zhou et al., 2018; Qu et al., 2023; Ghari et al., 2024). For this reason, any practical surveillance system intended for real environments must consider visual degradation, even if low-light enhancement is not the main contribution of the project.

Edge computing provides a practical direction for addressing some of these concerns. By processing data close to the camera, an edge-based system can reduce dependence on continuous cloud access and avoid sending every video frame to a remote server. Instead, the system can perform local detection and reasoning, then transmit only relevant alerts or event records. Abu Awwad et al. (2024) and Ali et al. (2024) emphasized that edge-based processing can reduce latency and bandwidth demand in surveillance applications. This

makes edge computing particularly useful where internet access is weak, expensive, or unreliable.

Edge Sentinel is developed within this context. It is a resilience-oriented edge-based intelligent visual surveillance prototype designed for infrastructure-constrained environments. The system performs local person detection, anonymous multi-object tracking, and rule-based behavioural-event reasoning on Raspberry Pi-class hardware. It uses configurable rules to identify selected security conditions such as restricted presence, lingering, crowd density, tailgating, and abnormal motion. The system also supports local event logging, event evidence, persistent alert queueing, HTTP alert recovery, and GSM/SMS alert capability.

The project does not attempt to build a facial recognition system or a general-purpose human activity recognition model. It also does not claim to solve all forms of video anomaly detection. Instead, it focuses on a practical and explainable approach. Persons are detected locally, anonymous track identifiers are used for continuity, and behavioural indicators are triggered using rules based on zones, duration, count, movement, and configured thresholds. This design makes the system easier to understand, tune, and evaluate. Privacy remains an important design consideration because the system avoids identity recognition under routine operation, but the main contribution of the work is resilience: maintaining useful surveillance functionality despite infrastructure limitations.

1.2 Research Motivation

The motivation for this project comes from the need for surveillance systems that remain useful under real deployment constraints. In many environments, security monitoring is required even when broadband internet is unavailable, power supply is unstable, and continuous human supervision cannot be guaranteed. A camera that only records footage may provide evidence after an incident, but it may not help users respond while the incident is taking place. Similarly, a smart camera that depends heavily on cloud processing may lose much of its value when connectivity becomes poor.

Nigerian-context studies make this concern more concrete. Nte et al. (2020) identified power supply problems, poor monitoring, lack of technicians, and malfunctioning equipment as practical barriers to effective CCTV use in Abuja. Aliyu and Airoboman (2025) also emphasized the need for low-cost security systems that can function in rural communities with limited internet access. These studies show that surveillance failure can come from infrastructure and operational weaknesses, not just from poor model accuracy.

This project is also motivated by the gap between academic computer vision research and deployment-aware surveillance design. Many studies focus on detection accuracy, anomaly

recognition, low-light enhancement, or model efficiency. These areas are important, but they do not fully address the complete system problem faced in constrained environments. A practical surveillance system must consider local processing, event logging, fallback communication, configuration, evidence handling, and recovery after communication failure. It must also be simple enough to run on affordable hardware.

Edge Sentinel addresses this motivation by combining lightweight local perception with rule-based behavioural reasoning and resilience-focused alert handling. The system uses detection and anonymous tracking to observe persons in a scene, then applies configurable rules to determine whether a security event has occurred. Instead of relying on a heavy temporal anomaly model for all behaviours, it uses explainable indicators that can be evaluated directly. This makes the system more suitable for Raspberry Pi-class deployment and easier to defend in practical security contexts.

The project is further motivated by the need for affordable, modular, and locally adaptable surveillance technology. Schools, hostels, laboratories, farms, storage areas, and small facilities may not be able to rely on enterprise surveillance infrastructure, stable broadband, or continuous cloud subscriptions. Edge Sentinel is therefore designed as a modular edge prototype that can be configured for different locations and security rules. Its purpose is not to replace large commercial systems, but to demonstrate a practical resilience-oriented architecture for environments where conventional surveillance systems may be fragile.

1.3 Research Aim and Objectives

The aim of this project is to develop and evaluate a resilient edge-based intelligent visual surveillance prototype that performs local person detection, anonymous tracking, rule-based behavioural-event reasoning, and persistent alert handling for infrastructure-constrained environments.

The specific objectives of the project are as follows:

1. To design an edge-based surveillance architecture that performs local person detection, anonymous tracking, and rule-based behavioural-event reasoning without requiring continuous cloud connectivity.
2. To implement configurable and explainable rule-based indicators for restricted presence, lingering, crowd density, tailgating, and abnormal motion using local detection, tracking, zone, duration, count, transition, and threshold data.

3. To develop a local event-logging and alert-delivery mechanism that persistently queues failed HTTP alerts, retries delivery after endpoint recovery, and supports GSM/SMS notification through an AT-command modem.
4. To evaluate the prototype using detector-tracker pair screening, labelled behavioural-event scenario outcomes, sustained Raspberry Pi resource profiling, and alert retention and recovery during a simulated HTTP endpoint outage.

1.4 Methodology

This project adopts an iterative prototype-based development methodology. The methodology is suitable because Edge Sentinel combines computer vision, embedded hardware, sensor input, tracking, event reasoning, alert communication, local storage, and a user-facing configuration interface. These components could not be fully specified and validated in a single linear process. The system therefore required repeated cycles of design, implementation, testing, evaluation, and refinement.

The first stage of the methodology involved problem identification and requirements analysis. The project problem was defined around the limitations of surveillance systems in environments where power supply, internet connectivity, and continuous human monitoring cannot be assumed. Functional and non-functional requirements were identified, including video capture, local person detection, tracking, event reasoning, zone configuration, event logging, alert delivery, SMS capability, and operation on Raspberry Pi-class hardware.

The second stage involved literature review and technology study. Relevant works were reviewed across intelligent visual surveillance, low-light computer vision, object detection, video anomaly detection, edge AI deployment, model optimization, and Nigerian infrastructure-related surveillance challenges. This review guided the decision to pursue an edge-based design that combines lightweight perception with explainable behavioural-event rules rather than relying entirely on cloud processing or complex temporal anomaly models.

The third stage involved system architecture design. Edge Sentinel was designed as a modular system with sensing, perception, configuration, behavioural reasoning, evidence/logging, communication, and local interface layers. The sensing layer includes the camera and PIR sensor. The perception layer includes the detector and tracker. The reasoning layer evaluates configured behavioural indicators. The communication layer manages alerts, persistent queueing, HTTP retry, and GSM/SMS capability. This modular structure allows individual parts of the system to be tested, replaced, or improved without redesigning the entire system.

The fourth stage involved prototype implementation and hardware integration. The implementation was developed around a Raspberry Pi 5, an IMX219 NoIR CSI camera, an HC-SR501 PIR sensor, and a SIMCOM A7670G modem. Software modules were implemented for frame processing, detection, tracking, indicator evaluation, zone configuration, event generation, alert queueing, and local dashboard interaction. Hardware components were verified functionally, while formal evaluation focused on the detector-tracker pipeline, behavioural scenarios, resource profiling, and alert queue recovery.

The fifth stage involved detector-tracker screening and final pipeline selection. Lightweight detection and tracking options were examined with the goal of supporting local surveillance reasoning on edge hardware. EfficientDet-Lite0 with ByteTrack became the selected reference detector-tracker pair. YOLOv8n with ByteTrack was later used only as a sensitivity experiment for a difficult running-past case and did not replace the selected reference pair.

The final stage involved system evaluation. The prototype was evaluated using labelled behavioural-event scenarios, sustained Raspberry Pi profiling, and alert recovery testing. The behavioural evaluation tested whether the implemented indicators produced the expected event outcomes across selected scenarios. The profiling evaluation examined processing performance and device stability under a saved-video workload. The alert recovery evaluation tested whether alerts generated during a simulated HTTP endpoint outage were retained and delivered after recovery. These evaluations provide evidence for the system's functionality while also identifying limitations that remain for future work.

1.5 Scope of the Study

The scope of this study is limited to the development and evaluation of an edge-based intelligent visual surveillance prototype for detecting selected security-relevant behaviours in infrastructure-constrained environments. The system focuses on local person detection, anonymous multi-object tracking, configurable zone-based monitoring, rule-based behavioural-event reasoning, event logging, evidence handling, persistent alert queueing, HTTP alert recovery, and GSM/SMS notification capability.

The implemented behavioural indicators are restricted presence, lingering, crowd density, tailgating, and abnormal motion. These indicators were selected because they represent practical security conditions that can be expressed using detection, tracking, zone, count, duration, transition, and movement information. The system does not attempt to identify individuals, perform facial recognition, or classify all possible human activities. It also does not claim to replace human security personnel, but to support faster awareness and more reliable event preservation.

The hardware scope of the project includes Raspberry Pi-class edge deployment, IMX219 NoIR camera input, HC-SR501 PIR motion sensing, and SIMCOM A7670G GSM/SMS communication. The software scope includes the perception pipeline, tracker integration, rule-based indicator logic, local configuration interface, event records, alert delivery, and persistent queue recovery. The evaluation scope covers detector-tracker screening, behavioural scenario outcomes, sustained Raspberry Pi saved-video profiling, hardware component verification, and simulated HTTP endpoint outage recovery.

The study does not include full cloud platform deployment, multi-device account management, continuous cloud synchronization, formal detector mAP evaluation, formal tracking metrics such as MOTA or IDF1, measured electrical power consumption in watts, solar/battery endurance testing, guaranteed SMS reliability, or complete pitch-dark surveillance validation. These areas are outside the completed scope and are treated as possible directions for future work.

1.6 Significance of the Study

This study is significant because it addresses a practical surveillance problem that is common in infrastructure-constrained environments. Many conventional CCTV systems depend on continuous human monitoring or later review of recorded footage. In places where power supply is unstable, internet connectivity is weak, and technical support is limited, such systems may fail to provide timely and reliable security awareness. Edge Sentinel responds to this problem by placing surveillance intelligence closer to the camera and allowing important event reasoning to happen locally.

The project contributes to intelligent surveillance by demonstrating a practical edge-based architecture that combines person detection, anonymous tracking, configurable behavioural indicators, local event logging, evidence handling, persistent alert queueing, and fallback SMS capability. This shows that useful surveillance intelligence does not always need to depend on continuous cloud processing or heavy video anomaly detection models. By using explainable rules, the system can generate alerts with clearer trigger reasons, making it easier to understand and evaluate its behaviour.

The study is also significant to Nigerian and similar deployment contexts where surveillance systems are affected by unreliable power, poor maintenance, weak connectivity, and affordability concerns. Instead of treating these conditions as external problems, the project includes them as part of the design motivation. The alert queue recovery mechanism, local processing, and GSM/SMS capability support the idea that a surveillance system should continue preserving useful event information even when remote delivery is temporarily unavailable.

For future researchers and developers, the project provides a foundation for improving edge-based surveillance systems. The prototype can be extended through stronger low-light evaluation, improved abnormal-motion handling, formal detector and tracker benchmarking, measured power profiling, solar or battery testing, and wider field trials. The work therefore contributes both a working prototype and a clear direction for building more resilient surveillance systems for constrained environments.

1.7 Organization of the Project

This project report is organized into five chapters.

Chapter One introduces the background of the study, research motivation, aim and objectives, methodology, and organization of the report.

Chapter Two presents the literature review. It discusses intelligent visual surveillance, low-light computer vision, object detection, video anomaly detection, edge AI deployment, infrastructure-constrained security systems, and the research gaps that motivate Edge Sentinel.

Chapter Three presents the methodology and system design. It describes the iterative prototype-based development model, system requirements, architecture, hardware design, software design, detector-tracker selection, behavioural indicator design, alert resilience design, local interface, evaluation methodology, and ethical considerations.

Chapter Four presents the implementation, results, and discussion. It describes the implemented prototype, hardware verification, detector-tracker selection, behavioural scenario evaluation, running-past limitation, YOLOv8n sensitivity experiment, Raspberry Pi resource profiling, alert queue recovery evaluation, and discussion of findings.

Chapter Five presents the conclusion and recommendations. It summarizes the work carried out, highlights the contribution of the project, discusses limitations, and recommends areas for further development and research.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews literature related to intelligent visual surveillance, low-light computer vision, object detection, multi-object tracking, video anomaly detection, edge AI deployment, and infrastructure-constrained security systems. The purpose of the review is to establish the academic and technical foundation for Edge Sentinel and to identify the gaps that motivate the proposed system.

The review is organized into conceptual and related-work sections. The conceptual review explains the major ideas that support the project, including intelligent visual surveillance, edge computing, behavioural event reasoning, tracking-based surveillance, and infrastructure-constrained deployment. The related-work review then examines existing studies across three main categories: low-light vision and object detection, video anomaly detection and behavioural surveillance, and edge AI deployment for constrained environments. The chapter concludes by synthesizing the gaps identified in the reviewed works and showing how Edge Sentinel responds to them.

The reviewed literature shows that significant progress has been made in object detection, low-light enhancement, anomaly detection, model optimization, and edge deployment. However, many existing works focus mainly on model accuracy, detection performance, or isolated deployment experiments. Fewer studies address the complete system-level problem of building a surveillance architecture that can operate under unreliable power, weak connectivity, limited compute resources, and reduced human monitoring. This gap is especially important in Nigerian and similar infrastructure-constrained environments where conventional surveillance systems may be affected by power instability, cost, maintenance challenges, and internet dependence.

Edge Sentinel is positioned within this gap. The system is not designed as a facial recognition system or a full general-purpose human activity recognition model. Instead, it focuses on resilient edge-based surveillance using local person detection, anonymous multi-object tracking, configurable rule-based security indicators, local event logging, privacy-conscious evidence handling, and persistent alert delivery.

2.2 Conceptual Review

2.2.1 Intelligent Visual Surveillance

Visual surveillance refers to the use of cameras and video systems to monitor physical environments for safety, security, access control, and incident review. Traditional closed-circuit television systems mainly provide video feeds for live viewing or recorded footage for later investigation. Although useful, this approach is largely passive because the system does not independently understand the scene or notify users when security-relevant events occur. It depends heavily on human operators who may become distracted, fatigued, or unable to monitor multiple feeds continuously.

Intelligent visual surveillance improves on passive monitoring by applying computer vision and machine learning techniques to automatically detect objects, recognize activities, and identify abnormal or suspicious events. In such systems, video frames are not only stored but also analyzed to extract useful information. For example, an intelligent surveillance system may detect human presence, count people in an area, identify movement across a restricted boundary, or raise an alert when a person remains in a location longer than expected.

The literature shows that intelligent surveillance can reduce the burden of manual monitoring and support faster response to security events. Ali et al. (2024) noted that large volumes of surveillance footage are redundant and that manual screening can lead to missed events and human fatigue. Duong et al. (2023) similarly emphasized the need for automated anomaly detection in video surveillance because abnormal events are rare and difficult to identify manually. These observations support the need for surveillance systems that move beyond passive video recording toward active event detection.

For Edge Sentinel, intelligent visual surveillance is approached through a lightweight and explainable design. The system uses person detection as the perception layer, anonymous tracking for temporal continuity, and rule-based security indicators as the event reasoning layer. This allows the system to generate alerts based on defined security conditions, such as restricted presence, lingering, crowd density, tailgating, and abnormal motion, instead of relying only on continuous human monitoring.

2.2.2 Edge Computing in Surveillance

Edge computing refers to the processing of data close to where it is generated instead of sending all data to a remote cloud server. In surveillance systems, edge computing allows video frames to be processed on or near the camera device. This reduces the need for continuous video streaming, lowers bandwidth usage, and can improve response time because important decisions are made locally.

Cloud-based surveillance can provide high computational capacity, centralized storage, and remote access. However, it also introduces limitations. Continuous video streaming consumes bandwidth and may become unreliable when internet connectivity is weak. Cloud-based processing can also introduce latency, making it less suitable for time-sensitive security events. Abu Awwad et al. (2024) and Ali et al. (2024) identified latency and bandwidth demand as major concerns in cloud-dependent surveillance systems.

Edge-based surveillance is especially relevant for environments where internet access is unreliable or expensive. By performing core inference locally, a system can continue detecting relevant events even when cloud access is unavailable. Instead of transmitting every frame, it can send only event messages, snapshots, or lightweight alerts. This makes edge computing a suitable foundation for infrastructure-constrained surveillance.

However, edge computing also introduces constraints. Low-cost devices such as Raspberry Pi boards have limited CPU capacity, memory, storage, and power availability. Heavy detection or anomaly recognition models may perform well in cloud or GPU environments but may not be practical for local deployment on low-cost edge hardware. For this reason, Edge Sentinel's design emphasizes lightweight detection, anonymous tracking, deterministic event rules, and bounded evaluation rather than depending on a single heavy model for all surveillance intelligence.

2.2.3 Object Detection and Multi-Object Tracking

Object detection is one of the main foundations of intelligent visual surveillance. It allows a system to identify objects of interest in a frame, such as persons, vehicles, or animals. In the context of Edge Sentinel, person detection is the primary perception task because the behavioural indicators depend on knowing whether people are present in a scene and where they are located.

Detection alone, however, is not enough for many surveillance rules. A single frame can show that a person is present, but it cannot reliably show how long the person has remained in a zone, whether the person crossed from one zone to another, or whether movement was sustained over time. This is where multi-object tracking becomes important. Tracking connects detections across frames and assigns temporary track identifiers to detected persons. These identifiers do not represent real human identity, but they provide continuity for event reasoning.

Multi-object tracking is useful for indicators such as lingering, tailgating, abnormal motion, and movement across configured zones. A lingering rule requires the system to know that the same tracked person remained in an area for a given duration. A tailgating rule requires

transition patterns between zones or entry boundaries. Abnormal motion requires movement or speed evidence across frames rather than a single isolated detection. For these reasons, Edge Sentinel uses anonymous tracking as part of its perception pipeline.

ByteTrack is relevant to this design because it is a tracking approach that associates detection boxes across frames and is designed to improve tracking continuity by considering both high-confidence and lower-confidence detections during association. This is useful in practical surveillance scenes where detections may fluctuate due to motion, occlusion, lighting variation, or detector uncertainty. In Edge Sentinel, ByteTrack is used for anonymous continuity, not for identity recognition.

2.2.4 Behavioural Event Reasoning

Behavioural event reasoning refers to the process of converting low-level observations into meaningful security events. In visual surveillance, object detection may identify that a person is present in a frame, while tracking may show continuity across frames. Event reasoning adds interpretation by considering location, time, movement, count, duration, and configured rules.

There are different approaches to behavioural reasoning in surveillance. Some systems use deep learning models that learn temporal patterns from video sequences, such as CNN-LSTM, Bi-LSTM, optical flow, or 3D convolutional networks. These approaches can capture complex motion and activity patterns, but they may require high computation, large datasets, and careful training. They may also be difficult to interpret when an alert is generated.

An alternative approach is rule-based reasoning. In this approach, the system uses simple but meaningful conditions to identify events. A restricted-presence event may be triggered when a tracked person overlaps a configured zone. A lingering event may be triggered when a tracked person remains in an area beyond a configured duration. A crowd-density event may be triggered when the number of persons in a monitored zone exceeds a threshold. Tailgating and abnormal-motion indicators can also be represented through transition and movement rules.

Edge Sentinel adopts this rule-based approach. The decision is motivated by the resource limitations of edge devices and the need for explainable event alerts. Rather than using a heavy temporal anomaly model for all behaviours, the system uses local detection, anonymous tracking, zone information, and configurable rules to generate practical security events. This does not eliminate the possibility of future advanced temporal models, but it provides a realistic and interpretable foundation for Raspberry Pi-class deployment.

2.2.5 Infrastructure-Constrained Deployment

Infrastructure-constrained deployment refers to environments where reliable power, stable internet connectivity, continuous technical support, and enterprise-grade hardware cannot be assumed. In such environments, a surveillance system must be designed not only for detection accuracy, but also for practical operation under power, network, cost, and maintenance limitations.

This issue is particularly important in Nigerian and similar contexts. Nte et al. (2020), in a study of CCTV use in Abuja, identified practical challenges such as erratic power supply, power fluctuation, lack of technical support, malfunctioning systems, and poor monitoring practices. These issues reduce the effectiveness of conventional CCTV systems even when cameras are installed. Aliyu and Aioboman (2025) also emphasized that many advanced surveillance solutions are too expensive, power-intensive, or dependent on internet infrastructure for rural Nigerian communities.

These findings show that local surveillance problems are not only algorithmic. They are also infrastructural and operational. A model may achieve high accuracy in a dataset, but the complete system may still fail if it requires stable broadband, continuous grid power, or constant human monitoring. Therefore, infrastructure-constrained surveillance requires a system-level design that combines local inference, local storage, fallback communication, power-conscious operation, and simple configuration.

Edge Sentinel treats these constraints as part of the design problem. The system is designed around edge-first processing, local event logging, configurable rules, persistent alert queueing, and GSM/SMS alert capability. Where network access is available, alerts may be delivered through network-based channels. Where connectivity is weak or the remote endpoint is unavailable, local processing and alert retention can continue until delivery is possible.

2.3 Review of Related Works

This section reviews existing works related to the technical foundation of Edge Sentinel. The review is organized into three major categories. The first category covers low-light vision and object detection for surveillance. The second category covers video anomaly detection and behavioural surveillance. The third category covers edge AI deployment and infrastructure-constrained security systems.

2.3.1 Low-Light Vision and Object Detection for Surveillance

Low-light conditions are a major challenge in visual surveillance because security monitoring often occurs at night or in poorly illuminated areas. Under low illumination, images may suffer from poor contrast, noise, blur, weak edges, and loss of detail. These degradations can reduce both human visibility and machine detection accuracy. For surveillance systems that depend on object or person detection, poor visual quality can lead to missed detections, false detections, and unreliable event reasoning.

Several studies have investigated low-light image enhancement and object detection under degraded illumination. Li et al. (2022) reviewed deep learning-based low-light image and video enhancement methods and showed that enhancement research has moved from human-perception-driven enhancement toward machine-vision-oriented enhancement. This distinction is important because images that look visually pleasing to humans may not necessarily improve detection accuracy for a computer vision model.

Other works focus more directly on preserving features needed for detection. Hashmi et al. (2023) proposed FeatEnHancer, a plug-and-play feature enhancement module for object detection under low-light conditions. The study showed that enhancing hierarchical detection features can improve performance while avoiding some limitations of applying low-light enhancement as a separate preprocessing stage. Qu et al. (2023) proposed a real-time low-light enhancement method for ultra-high-definition transportation surveillance, showing that low-light enhancement can be optimized for speed and edge preservation.

Object detection models are central to Edge Sentinel because the system relies on person detection as the foundation for tracking and event reasoning. Tan et al. (2020) proposed EfficientDet, a scalable object detection architecture designed to improve the trade-off between accuracy and computational efficiency. EfficientDet is relevant to this project because its lightweight variants support the broader idea of efficient detection under constrained computation. Abu Awwad et al. (2024) demonstrated a low-light edge detection pipeline using enhancement selection and YOLOv5-tiny on smart camera platforms, showing that low-light detection can be performed locally with acceptable response time.

For Edge Sentinel, these works support two main design decisions. First, low-light conditions must be considered during system evaluation because surveillance environments are rarely ideal. Second, lightweight object detection is a practical foundation for edge surveillance. The current project uses EfficientDet-Lite0 as the selected reference detector, paired with ByteTrack for anonymous tracking. This choice is not presented as a universal best detector, but as a practical detector-tracker reference pair for the evaluated edge prototype.

Table 2.1: Low-Light Vision and Object Detection Literature Comparison

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
Abu Awwad et al. (2024)	Anomaly detection on edge smart cameras under low-light conditions	Random Forest classifier selects low-light enhancement model, followed by YOLOv5-tiny detection	ExDark dataset; Raspberry Pi 4, Jetson Nano, Intel NCS2	Approximately one-second edge response time; enhancement improved detection under low-light conditions	Dataset labeling limitations and limited unique samples affected evaluation	Demonstrates feasibility of low-light edge detection and supports event-triggered edge processing
Li et al. (2022)	Survey of deep learning-based low-light image and video enhancement	Taxonomy of supervised, unsupervised, and zero-shot enhancement methods	LOL, SID, DARK FACE, ExDark, LLIV-Phone and related datasets	Identified lightweight methods such as Zero-DCE and highlighted machine-vision-driven enhancement needs	Gap remains between visually pleasing enhancement and enhancement that improves machine vision tasks	Supports the need to treat low-light robustness as a machine-vision problem, not only an image appearance problem
Hashmi et al. (2023)	FeatEnHancer for object detection under low-light vision	Multi-headed attention and hierarchical feature enhancement for detection networks	ExDark, DARK FACE, ACDC, DarkVision	Improved detection performance on low-light datasets with a small	Improvement varies by task and may require task-specific optimization	Shows that preserving detection-relevant features is important under

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
				parameter footprint		poor lighting
Tan et al. (2020)	EfficientDet: scalable and efficient object detection	Weighted BiFPN and compound scaling of resolution, depth, and width	MS COCO, Pascal VOC	EfficientDet-D0 achieved competitive AP with fewer FLOPs than many larger detectors	Small-object detection and scaling choices may affect performance	Supports efficient detector selection and provides background for considering EfficientDet-based models
Qu et al. (2023)	Real-time low-light enhancement for UHD transportation surveillance	DDNet using color and gradient domain processing with gradient enhancement	LOL, DICM, LIME, MEF, TMDIED	Demonstrated high-speed enhancement for high-resolution surveillance images	Gradient methods may be affected by severe sensor noise	Shows that real-time enhancement is possible, but may still add complexity to edge deployment
Tran et al. (2024)	Low-light enhancement framework for object detection in fisheye datasets	Multi-stage enhancement and detection framework using NAFNet, GSAD, and detector ensemble	FishEye8K, Objects365	Improved low-light fisheye detection performance in AI City Challenge context	Multi-stage ensemble pipeline is computationally heavy for low-cost edge devices	Demonstrates the value of enhancement for detection but highlights why simpler edge approaches are needed

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
Ghari et al. (2024)	Survey of pedestrian detection in low-light conditions	Review of handcrafted, deep learning, and multispectral fusion methods	KAIST, FLIR, LLVIP, OSU, SCUT, NightOwls	Identified progress in nighttime pedestrian detection and multispectral approaches	Heavy dependence on benchmark datasets; limited real-world data in many studies	Supports the need for real-world evaluation because low-light detection may not generalize well
Wu (2023)	Image enhancement for improving edge AI accuracy in dark light	Median/Wiener filtering with CNN-based recognition	Edge Impulse platform with simulated dark images	Reported improved recognition confidence under dark-light conditions	Limited task scope and reliance on simulated darkness	Supports low-cost software enhancement as a possible future path for improving edge detection under poor lighting

2.3.2 Video Anomaly Detection and Behavioural Surveillance

Video anomaly detection is a major area of intelligent surveillance research. It focuses on identifying unusual, suspicious, or abnormal events in video streams. Such events may include fighting, accidents, intrusions, abnormal crowd movement, abandoned objects, unauthorized presence, or unusual activity patterns. Unlike simple object detection, anomaly detection usually requires some understanding of motion, time, context, and expected behaviour within a scene.

Several studies have explored deep learning-based approaches for detecting abnormal activity in video surveillance. These approaches often use temporal models such as Long Short-Term Memory networks, Bi-LSTM, 3D convolutional networks, optical flow, future-frame prediction, or weakly supervised learning. These methods can capture complex patterns in human motion and scene dynamics, making them useful for large-scale video anomaly detection tasks. However, they also introduce higher computational cost and may require large datasets, GPU acceleration, and careful training.

Ali et al. (2024) proposed an edge-computing-enabled abnormal activity recognition system using a hybrid Inception-CNN and Bi-LSTM model. Their work is relevant because it demonstrates that temporal reasoning can be optimized for Raspberry Pi-class deployment through model quantization. However, the model still depends on training with selected anomaly classes and may be affected by visual similarity between activities or camera movement. Similarly, Nejad and Haque (2024) used a two-stream I3D network for weakly supervised anomaly detection, combining appearance and motion information. Although this approach improves anomaly recognition, it is computationally expensive because each frame is processed through multiple streams.

Other studies have focused on crowd-related anomaly detection. Nasir et al. (2025) proposed a YOLOv8-based framework for abnormal behaviour detection in dense crowds, using Soft-NMS to improve detection under occlusion. This is relevant to surveillance in crowded spaces, but the additional computational load may require optimization before deployment on low-cost edge devices. Batzner et al. (2023) proposed EfficientAD for visual anomaly detection with millisecond-level latency, showing that lightweight anomaly detection is possible in industrial inspection contexts. However, industrial visual anomaly detection differs from open-environment human surveillance because the scene structure, object categories, and anomaly definitions are more controlled.

Survey works also highlight the challenges of video anomaly detection. Duong et al. (2023) classified anomaly detection methods into reconstruction-based, classification-based,

prediction-based, and scoring-based approaches. The survey shows that temporal modelling can improve anomaly detection, but methods such as optical flow and 3D convolution can be computationally demanding. Tusher et al. (2025) also emphasized the importance of interpretability in surveillance systems, especially where alerts must be verified and acted upon.

These studies show that full video anomaly detection is technically valuable but may be too complex for the Edge Sentinel prototype scope. Therefore, Edge Sentinel adopts a simpler and more explainable approach. It uses local person detection and anonymous tracking as the foundation, then applies deterministic behavioural rules to generate security events. This allows the system to evaluate practical indicators such as restricted presence, lingering, crowd density, tailgating, and abnormal motion without depending on a heavy general-purpose anomaly model.

Table 2.2: Video Anomaly Detection and Behavioural Surveillance Literature Comparison

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
Ali et al. (2024)	Edge-computing-enabled abnormal activity recognition for visual surveillance	Hybrid Inception-CNN and Bi-LSTM model with TFLite quantization	UCF-Crime subset; Raspberry Pi 4	Reported 80.92 percent accuracy and 32.21 FPS on Raspberry Pi 4 after quantization	Limited anomaly classes; misclassification may occur due to camera movement or visual similarity between activities	Demonstrates feasibility of optimized temporal activity recognition on Raspberry Pi-class hardware, but also shows the complexity of full abnormal activity recognition
Nasir et al. (2025)	Real-time dense crowd abnormal behaviour detection using YOLOv8	YOLOv8-based CADF framework with Soft-NMS for dense crowd occlusion handling	Hajjv2, UCSD, ShanghaiTech	Reported 91.6 percent accuracy, 0.947 mAP50, and 46 FPS in evaluated settings	Soft-NMS increases computational load; further optimization is required for edge deployment	Supports the relevance of YOLO-based detection for crowd-related indicators and informs Edge Sentinel’s crowd-density monitoring direction

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
Batzner et al. (2023)	EfficientAD for accurate visual anomaly detection at millisecond-level latency	Student-teacher approach using Patch Description Network and calibrated autoencoder	MVTec AD, VisA, MVTec LOCO	Reported millisecond-level inference and high AU-ROC in industrial anomaly detection tasks	Designed mainly for industrial anomaly detection and controlled scenes	Shows that lightweight anomaly detection is possible, but its industrial focus differs from open-environment security surveillance
Duong et al. (2023)	Survey of deep learning-based anomaly detection in video surveillance	Review of reconstruction, classification, future-frame prediction, and scoring-based anomaly detection methods	Multiple benchmark datasets including UMN, CASIA, UCF-Crime and others	Identified LSTM, 3D CNN and related temporal methods as important for capturing motion dynamics	Optical flow and temporal models can be computationally expensive and power-hungry for edge devices	Provides theoretical support for behavioural reasoning while justifying the need for lighter alternatives on constrained devices
Nejad & Haque (2024)	Weakly supervised anomaly detection using two-stream I3D network	Two-stream I3D network using RGB and optical-flow features with Multiple Instance	UCF-Crime	Reported 85.41 percent AUC for dual-stream configuration	High computational and memory cost due to dual-stream processing	Demonstrates the strength of motion-aware anomaly detection but also highlights why

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
		Learning				Edge Sentinel avoids heavy two-stream models in the prototype
Liu et al. (2018)	Future-frame prediction for anomaly detection	U-Net-based future frame prediction and comparison with actual frames	CUHK Avenue, UCSD Ped2, ShanghaiTech	Reported strong anomaly detection performance, including 95.4 percent AUC on UCSD Ped2	Requires careful implementation and GPU support for real-time performance	Supports the idea that deviations from expected behaviour are useful, but the method is too heavy for the current edge-focused prototype
Tusher et al. (2025)	Computer vision anomaly detection and class distinction	MobileNetV2-based CNN with OpenCV and explainability analysis	Custom high-security dataset	Reported high classification accuracy for intruder/admin classes and real-time performance in the evaluated setting	Limited dataset size and less emphasis on extreme resource-constrained deployment	Supports the importance of interpretable surveillance and lightweight models for security classification tasks

2.3.3 Edge AI and Infrastructure-Constrained Security Systems

Edge AI deployment is an important research area for intelligent surveillance because real-world security systems are not limited by model accuracy alone. They must also operate under constraints such as limited compute power, memory, storage, energy, bandwidth, and maintenance capacity. This is particularly important for surveillance systems intended for rural, semi-urban, institutional, or off-grid environments.

A major motivation for edge surveillance is the limitation of cloud-dependent video analytics. Cloud processing can provide strong computational resources, but it requires stable network connectivity and may introduce delay or bandwidth cost. Abu Awwad et al. (2024) showed that edge-based low-light anomaly detection can reduce the latency associated with cloud-hybrid approaches. Ali et al. (2024) also argued that large volumes of surveillance footage are redundant and that local edge processing can reduce unnecessary transmission while improving responsiveness.

Several works focus on optimizing models for low-resource devices. Cheng et al. (2018) reviewed model compression and acceleration techniques, including pruning, parameter sharing, low-rank factorization, compact filters, and knowledge distillation. These methods are important because deep neural networks are often too large for deployment on constrained hardware. Gholami et al. (2021) reviewed quantization methods and showed that integer-based inference can significantly improve energy efficiency and reduce computational demands. These works provide a technical foundation for future Edge Sentinel optimization, including possible TFLite conversion, quantization, and lightweight model benchmarking.

The Nigerian deployment context adds another layer of importance to edge AI research. Nte et al. (2020) reported that CCTV systems in Abuja face challenges such as erratic power supply, poor maintenance, lack of technical support, and limited operational effectiveness. This shows that surveillance failure can result from infrastructure and operational weaknesses, not just poor camera coverage. Aliyu and Airoboman (2025) addressed a related problem by proposing a low-cost AI-enabled smart security system for Nigerian rural communities using ESP32-CAM, PIR sensing, MobileNet classification, and GSM/SMS alerting. Their work is especially relevant because it validates the importance of affordable hardware and internet-independent alerting in Nigerian security contexts.

However, Aliyu and Airoboman's system is limited mainly to binary classification, while Edge Sentinel targets a richer edge-surveillance architecture. Edge Sentinel uses Raspberry Pi-class hardware to support local person detection, anonymous tracking, configurable zones,

rule-based behavioural indicators, local event logging, evidence handling, and persistent alert delivery. This places the system between very lightweight embedded security devices and heavier deep anomaly detection systems.

The reviewed edge AI literature therefore supports the design philosophy of Edge Sentinel. The system is built around the idea that surveillance intelligence should not depend entirely on continuous cloud connectivity. Instead, the most important functions should remain local, while alerts and evidence are handled through available communication channels. This makes the architecture more suitable for infrastructure-constrained deployment.

Table 2.3: Edge AI and Infrastructure-Constrained Security Systems Literature Comparison

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
Aliyu & Airoboman (2025)	AI-enabled smart security system for real-time threat detection in Nigerian rural communities	Quantized MobileNet for binary classification, integrated with ESP32-CAM, PIR sensor, and GSM module	Custom 226-sample dataset; ESP32-CAM hardware	Reported 93.3 percent accuracy, 60 to 77 ms classification time, and approximately 3.2 s average SMS alert delivery	Limited to binary human/animal classification; constrained by ESP32 hardware and limited real-world environmental testing	Provides a Nigerian regional baseline and validates the use of low-cost hardware, PIR sensing, and GSM/SMS alerting
Nte et al. (2020)	Challenges and prospects of ICTs in crime prevention and CCTV use in Abuja	Socio-technical review and field survey of CCTV use and effectiveness	CCTV infrastructure and institutional/public surveillance context in Abuja, Nigeria	Identified erratic power, poor monitoring, lack of technicians, malfunctioning equipment, and operational redundancy as key challenges	Does not propose or implement a new technical surveillance architecture	Provides local evidence that Nigerian surveillance systems require infrastructure-aware design
Ali et al. (2024)	Edge-computing-enabled abnormal activity	Hybrid Inception-CNN and Bi-LSTM optimized	Raspberry Pi 4; UCF-Crime subset	Reported real-time inference and improved energy	Limited anomaly classes and sensitivity to	Supports edge deployment feasibility and

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
	recognition for visual surveillance	using TFLite quantization		efficiency after quantization	visual similarity or camera motion	highlights the role of optimization for Raspberry Pi-class systems
Abu Awwad et al. (2024)	Edge smart-camera anomaly detection under low-light conditions	Event-triggered enhancement selection with Random Forest and YOLOv5-tiny detection	ExDark dataset; Raspberry Pi 4, Jetson Nano, Intel NCS2	Reported approximately one-second edge response and improved low-light detection performance after enhancement	Relies partly on additional acceleration hardware; dataset limitations affected evaluation	Supports edge-first and event- triggered surveillance design for constrained environments
Cheng et al. (2018)	Survey of model compression and acceleration for deep neural networks	Review of pruning, parameter sharing, low-rank factorization, compact filters, and knowledge distillation	Survey across embedded, mobile, FPGA and distributed deployment contexts	Showed that compression can reduce model size and computation significantly while preserving accuracy in some cases	Some compression methods require retraining and may be difficult to adapt across tasks	Provides an optimization roadmap for future Edge Sentinel model compression and acceleration
Gholami et al. (2021)	Survey of quantization methods for	Review of uniform/non- uniform	ARM Cortex-M, Edge TPU, mobile and commercial	Showed that integer quantization can	Mixed-precision and sub-INT8 tooling remain less	Supports possible INT8 quantization and efficient

Author/Year	Title/Focus	Methodology/ Model	Dataset/Platform	Key Results	Limitations	Relevance to Edge Sentinel
	efficient neural network inference	quantization, post-training quantization, and quantization-aware training	edge processors	improve energy efficiency and support low-power inference	mature in some deployment settings	inference paths for future Edge Sentinel optimization
Tan et al. (2020)	EfficientDet: scalable and efficient object detection	EfficientDet architecture using BiFPN and compound scaling	MS COCO	EfficientDet-D0 achieved competitive accuracy with significantly fewer FLOPs than larger detectors	Accuracy may vary with small-object detection and scaling choices	Supports efficient object detection as a design direction and provides background for EfficientDet-based detector selection
Wu (2023)	Improving edge AI accuracy using image enhancement in dark light	Median and Wiener filtering with CNN-based recognition	Edge Impulse platform with simulated dark images	Reported improved recognition confidence under dark-light conditions	Limited task scope and reliance on simulated darkness	Supports low-cost software enhancement as a possible future path for improving edge detection under poor lighting

2.4 Comparative Analysis of Reviewed Works

The reviewed literature shows that intelligent visual surveillance has developed across several related but distinct research directions. These include low-light image enhancement, object detection, tracking, video anomaly detection, edge AI optimization, and deployment of low-cost security systems. Each direction contributes useful knowledge to the development of Edge Sentinel, but none fully addresses the complete system-level problem targeted in this project.

Low-light vision studies such as Li et al. (2022), Ghari et al. (2024), Qu et al. (2023), Hashmi et al. (2023), and Tran et al. (2024) demonstrate that poor illumination can significantly affect computer vision performance. These works are useful because they show that surveillance systems must be evaluated under degraded visual conditions, not only under ideal lighting. However, many low-light enhancement methods introduce additional processing complexity, especially when used as a separate preprocessing stage before detection. For an edge system such as Edge Sentinel, low-light robustness must therefore be balanced against compute cost, latency, and power consumption.

Object detection studies provide the foundation for the perception layer of Edge Sentinel. EfficientDet, proposed by Tan et al. (2020), demonstrates how object detection models can be scaled for better efficiency using BiFPN and compound scaling. YOLO-based approaches, including the YOLOv5-tiny pipeline used by Abu Awwad et al. (2024), also show the practical value of lightweight real-time detection. These works support the use of lightweight detection models for edge surveillance. In the implemented prototype, EfficientDet-Lite0 became the selected reference detector, while YOLOv8n was used only as a sensitivity comparison in a difficult diagnostic case.

Tracking literature also supports the design of Edge Sentinel because many behavioural indicators require temporal continuity. A person must be linked across frames before the system can reason about lingering duration, zone transitions, tailgating, or sustained movement. ByteTrack is useful in this context because it provides anonymous track continuity from detection boxes without introducing identity recognition. This supports the privacy-conscious nature of the system while still enabling practical behavioural-event reasoning.

Video anomaly detection literature provides important insight into behavioural analysis. Studies such as Ali et al. (2024), Duong et al. (2023), Nejad and Haque (2024), Batzner et al. (2023), Liu et al. (2018), Nasir et al. (2025), and Tusher et al. (2025) show that abnormal behaviour can be detected using temporal models, reconstruction methods, future-frame

prediction, weak supervision, or classification-based approaches. However, many of these models are computationally demanding or require large datasets and GPU support. Some methods also behave like black boxes, making it difficult for users to understand why an alert was generated. This supports Edge Sentinel's decision to use explainable rule-based behavioural indicators.

Edge AI and optimization studies address the practical challenge of deploying machine learning models on low-power devices. Cheng et al. (2018) and Gholami et al. (2021) show that model compression, pruning, and quantization can reduce computational cost and improve deployment feasibility. Ali et al. (2024) also demonstrates that optimized models can run on Raspberry Pi hardware. However, optimization alone does not solve the full surveillance deployment problem. A complete field-ready system must also consider communication fallback, local alert retention, power constraints, event logging, configuration, and resilience when connectivity fails.

The Nigerian-context works are particularly important because they connect the technical problem to local deployment realities. Nte et al. (2020) shows that CCTV systems in Abuja may be affected by erratic power supply, lack of technical staff, poor monitoring, and system malfunction. Aliyu and Airoboman (2025) propose a low-cost AI-enabled security system for Nigerian rural communities using ESP32-CAM, PIR sensing, MobileNet, and GSM/SMS alerts. These works support the need for affordable and connectivity-independent surveillance solutions. However, Nte et al. (2020) does not propose a technical architecture, while Aliyu and Airoboman (2025) is limited mainly to binary classification on very low-cost hardware. Edge Sentinel extends this direction by proposing a Raspberry Pi-class architecture with local person detection, anonymous tracking, configurable zones, multi-indicator event reasoning, local logging, evidence handling, and persistent alert delivery.

Overall, the reviewed works provide strong support for individual parts of Edge Sentinel. Low-light studies support the need for robustness under poor illumination. Object detection studies justify lightweight model selection. Tracking supports temporal continuity without identity recognition. Anomaly detection studies motivate behavioural event reasoning while revealing the cost of heavy temporal models. Edge AI studies justify local inference and optimization. Nigerian-context studies justify the infrastructure-first framing of the project. The remaining gap is the integration of these ideas into a modular, resilient, edge-based surveillance architecture suitable for infrastructure-constrained environments.

Table 2.4: Research Gap Synthesis and Edge Sentinel Response

Identified Gap	Supporting Literature	Explanation of the Gap	Implication for Edge Sentinel	Edge Sentinel Response
Passive monitoring and human error	Ali et al. (2024); Duong et al. (2023); Elmetwally et al. (2025); Nte et al. (2020)	Traditional CCTV systems often depend on human operators to monitor video feeds or review footage after an incident. Manual monitoring is time-consuming and prone to fatigue, missed events, and delayed response.	A surveillance system should support active event detection rather than relying only on passive recording and human attention.	Edge Sentinel introduces local person detection, anonymous tracking, and rule-based event generation to identify selected security conditions and notify users when attention is required.
Low-light and degraded visual conditions	Li et al. (2022); Ghari et al. (2024); Zhou et al. (2018); Qu et al. (2023)	Low-light environments introduce noise, low contrast, blur, and weak visual features, reducing the reliability of object detection and surveillance analysis.	The system must be evaluated under realistic lighting conditions before assuming reliable deployment performance.	Edge Sentinel treats low-light operation as an important deployment requirement and motivates further evaluation using NoIR imaging, controlled illumination, and optional enhancement techniques where necessary.
Edge resource limitations	Cheng et al. (2018); Gholami et al. (2021); Ali et al. (2024)	Deep learning models can be computationally expensive and may require	Model selection must prioritize lightweight inference and practical	Edge Sentinel uses a lightweight reference pipeline based on

Identified Gap	Supporting Literature	Explanation of the Gap	Implication for Edge Sentinel	Edge Sentinel Response
		memory, processing power, and energy beyond what low-cost edge devices can provide.	deployment on Raspberry Pi-class hardware.	EfficientDet-Lite0 and ByteTrack, while keeping optimization paths such as TFLite conversion and quantization within future improvement scope where needed.
Need for temporal continuity	ByteTrack-related tracking literature; Duong et al. (2023); Nasir et al. (2025)	Many behavioural indicators require continuity across frames rather than isolated detections. Without tracking, it is difficult to reason about duration, movement, transitions, and repeated presence.	Surveillance rules such as lingering, tailgating, and abnormal motion require anonymous track continuity.	Edge Sentinel incorporates ByteTrack to maintain temporary track IDs for detected persons without performing identity recognition.
Computational cost of complex temporal models	Duong et al. (2023); Nejad & Haque (2024); Nasir et al. (2025)	Many video anomaly detection approaches use optical flow, 3D CNNs, two-stream networks, or other temporal models that are computationally expensive for low-power	Full general-purpose anomaly detection may be unsuitable for the current edge prototype on Raspberry Pi-class hardware.	Edge Sentinel uses lightweight person detection and anonymous tracking as the perception layer, then applies explainable rule-based indicators for selected

Identified Gap	Supporting Literature	Explanation of the Gap	Implication for Edge Sentinel	Edge Sentinel Response
		devices.		behavioural events.
Explainability and verification barriers	Tusher et al. (2025); Duong et al. (2023); Nasir et al. (2025)	Black-box anomaly detection models can generate alerts without clearly explaining why an event was classified as abnormal, making verification and tuning difficult.	Security alerts should include interpretable reasons so users can understand and verify system decisions.	Edge Sentinel uses configurable rule-based event logic. Events can include human-readable trigger reasons such as restricted-zone entry, duration threshold exceeded, or crowd threshold exceeded.
Cloud and bandwidth dependency	Abu Awwad et al. (2024); Ali et al. (2024)	Cloud-based surveillance can introduce latency, bandwidth cost, and service failure when connectivity is weak or unavailable. Continuous video streaming may not be practical in infrastructure-constrained environments.	The system should avoid depending on continuous internet access for core detection and event generation.	Edge Sentinel follows an edge-first design where core inference and event reasoning occur locally. Alerts can be sent through available network channels, while local logging and persistent queueing support continued operation when remote delivery fails.
Nigerian infrastructure and affordability constraints	Nte et al. (2020); Aliyu & Airoboman (2025)	Nigerian surveillance deployments may be affected by erratic power,	A locally relevant surveillance system should treat infrastructure	Edge Sentinel is designed around Raspberry Pi-class hardware, local processing,

Identified Gap	Supporting Literature	Explanation of the Gap	Implication for Edge Sentinel	Edge Sentinel Response
		poor maintenance, limited technical support, affordability constraints, and weak internet availability.	limitations as design constraints rather than afterthoughts.	configurable monitoring, event logging, persistent alert handling, and GSM/SMS capability.
Lack of integrated resilience-aware architecture	Abu Awwad et al. (2024); Ali et al. (2024); Aliyu & Airoboman (2025); Nte et al. (2020)	Existing works often focus on one part of the problem, such as detection, anomaly classification, low-light enhancement, tracking, or low-cost alerting, rather than a complete resilience-aware surveillance architecture.	There is a need for a modular system that combines perception, tracking, event reasoning, evidence handling, local logging, and adaptive alerting.	Edge Sentinel proposes a modular edge-based architecture with sensing, local inference, anonymous tracking, behavioural event reasoning, evidence/logging, communication, and user configuration layers designed to degrade gracefully under infrastructure constraints.

2.5 Summary of Identified Gaps

From the reviewed literature, several gaps can be identified. First, traditional CCTV and many surveillance systems remain dependent on human monitoring. This creates a passive security model where footage may only become useful after an incident has occurred. Intelligent surveillance research attempts to address this issue through automated detection and anomaly recognition, but many approaches still require significant computational resources or controlled deployment conditions.

Second, low-light and degraded visual conditions remain a major challenge. Surveillance environments are not always well illuminated, and image degradation can affect detection, tracking, and event reasoning. Although low-light enhancement methods have improved significantly, many enhancement pipelines introduce additional computational cost. For low-cost edge deployment, there is a need to balance robustness with processing efficiency.

Third, behavioural surveillance often requires temporal continuity. Single-frame detection is not enough for reasoning about duration, transition, crowd movement, or sustained motion. Tracking is therefore important, but the tracking approach must remain lightweight and privacy-conscious. This supports the use of anonymous tracking rather than identity-based recognition.

Fourth, many video anomaly detection methods are too computationally heavy for the current class of deployment targeted by this project. Techniques such as optical flow, 3D CNNs, two-stream networks, and future-frame prediction can capture complex behaviour, but they may be unsuitable for Raspberry Pi-class hardware without optimization or acceleration. This creates a need for lighter and more interpretable event reasoning methods in practical edge surveillance systems.

Fifth, explainability remains a concern. Security alerts are more useful when they include clear reasons that can be verified by users. A black-box anomaly score may be difficult to interpret, especially in site-specific environments where users need to understand why an event was triggered. Rule-based indicators provide a practical alternative by linking alerts to understandable conditions such as zone entry, duration threshold, crowd threshold, or movement pattern.

Sixth, deployment constraints in Nigerian and similar environments are underrepresented in much of the intelligent surveillance literature. The reviewed Nigerian-context studies show that power instability, poor maintenance, weak internet availability, cost, and lack of

technical support can reduce the effectiveness of surveillance systems. This means that surveillance design for such environments must consider more than detection accuracy.

Finally, the literature shows a gap in integrated resilience-aware surveillance architecture. Many works address a single component, such as low-light enhancement, object detection, tracking, anomaly recognition, model compression, or low-cost alerting. Fewer works combine local inference, anonymous tracking, explainable event reasoning, local evidence handling, persistent alert queueing, and fallback communication into one coherent system.

Edge Sentinel is designed to respond to these gaps. It combines lightweight local person detection, anonymous tracking, configurable behavioural indicators, event logging, evidence capture, and persistent alert handling within a modular edge-based architecture. The system is intended to remain useful even when internet connectivity is weak, remote alert delivery is interrupted, and human monitoring cannot be continuous.

2.6 Summary of Chapter

This chapter reviewed literature related to intelligent visual surveillance, low-light vision, object detection, multi-object tracking, video anomaly detection, edge AI deployment, and infrastructure-constrained security systems. The review showed that existing research has made important contributions in improving object detection, enhancing low-light images, recognizing abnormal behaviours, optimizing models for edge devices, and supporting low-cost security systems.

The chapter also showed that practical deployment remains a major challenge. Many high-performing surveillance models require substantial computation, stable infrastructure, or cloud support. In Nigerian and similar contexts, surveillance systems may also be affected by erratic power supply, limited internet access, cost, maintenance difficulties, and limited human monitoring. These challenges support the need for a resilient edge-based surveillance system.

The identified gaps motivate the design of Edge Sentinel. The proposed system focuses on local person detection, anonymous tracking, explainable rule-based event reasoning, configurable monitoring zones, local logging, privacy-conscious evidence handling, persistent alert queueing, and GSM/SMS capability. Chapter Three presents the methodology and system design used to develop the proposed system.

CHAPTER THREE

METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

This chapter presents the methodology and system design used in the development of Edge Sentinel. The chapter explains how the project was planned, designed, implemented, and evaluated. It also describes the architectural structure of the system, the hardware and software components, the detector-tracker pipeline, the behavioural-event reasoning approach, the alerting mechanism, and the evaluation strategy.

Edge Sentinel was developed as an edge-based intelligent visual surveillance prototype for infrastructure-constrained environments. The system was designed to perform local person detection, anonymous tracking, rule-based behavioural-event reasoning, local event logging, and persistent alert handling without depending on continuous cloud connectivity. The design was influenced by the need to build a system that is not only intelligent, but also practical under weak connectivity, limited compute resources, and unreliable infrastructure.

The chapter begins by describing the development methodology adopted for the project. It then presents the requirements' analysis, system architecture, hardware design, software design, detection and tracking design, behavioural indicator design, event and alerting design, operational state model, and evaluation methodology. Ethical and privacy considerations are also discussed because the system deals with visual surveillance and security-event evidence.

3.2 Research and Development Methodology

This project adopted an iterative prototype-based development methodology. This methodology was selected because Edge Sentinel combines several interacting components, including computer vision, embedded hardware, sensor input, multi-object tracking, behavioural-event logic, alert delivery, persistent queueing, local storage, and a web-based configuration interface. These components could not be fully designed, implemented, and validated in one fixed sequence. The project therefore required repeated cycles of design, implementation, testing, evaluation, and refinement.

A purely waterfall approach would have been too rigid for this project because several important design decisions depended on implementation evidence. For example, the final detector-tracker pair could only be selected after practical evaluation of lightweight detection and tracking options. The behavioural indicators also required testing with scenario videos before their limitations could be understood. Similarly, hardware components such as the

camera, PIR sensor, and GSM modem had to be verified through direct testing before they could be described confidently as part of the prototype.

The iterative prototype-based approach allowed the system to evolve from an initial concept into a more complete evaluated prototype. Early stages focused on defining the problem, reviewing literature, identifying requirements, and designing a resilience-oriented architecture. Later stages focused on implementing the perception pipeline, adding tracking, configuring behavioural indicators, verifying hardware components, evaluating scenario outcomes, profiling Raspberry Pi performance, and testing alert queue recovery. Each stage produced feedback that influenced the next version of the system.

The methodology also matches the practical nature of the project. Edge Sentinel is not only a software application. It is a cyber-physical surveillance prototype that depends on the interaction between hardware, software, AI models, sensors, communication channels, and deployment constraints. For such a system, the development process must allow testing and correction as soon as real constraints appear. The final prototype therefore reflects both the original design goals and the lessons learned during implementation.

3.3 Iterative Prototype-Based Development Model

The development model used for this project is shown in Figure 3.1. The model begins with problem identification and requirements analysis. This is followed by literature review and technology study, where existing works on intelligent surveillance, low-light vision, object detection, tracking, edge AI, anomaly detection, and infrastructure-constrained security systems were examined. These activities informed the initial system architecture.

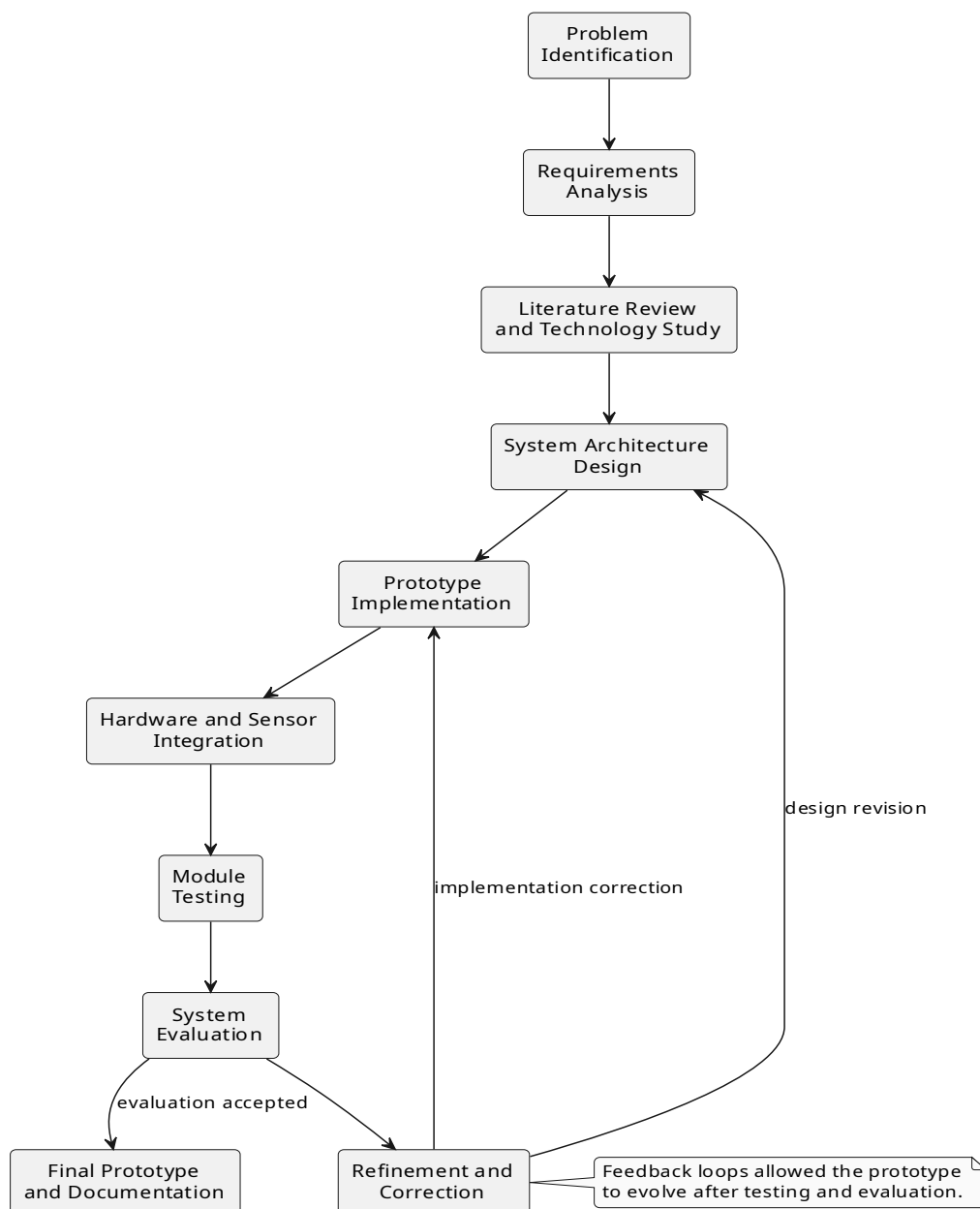
After the initial architecture was designed, the system entered the implementation phase. The prototype was built in modules so that individual parts could be tested and improved independently. The camera pipeline, detector, tracker, behavioural indicators, alert manager, persistent queue, local configuration interface, PIR input, and SMS capability were developed and verified through repeated implementation cycles.

Testing and evaluation were not treated as activities that occurred only at the end of the project. Instead, they were used throughout the development process to guide refinement. For example, evaluation of detector-tracker behaviour led to the adoption of EfficientDet-Lite0 with ByteTrack as the selected reference pipeline. Scenario evaluation also revealed that the abnormal-motion running-past case produced a false negative because the track duration was too brief to satisfy the sustained-duration requirement. This result was not hidden or overfitted away. It was treated as a limitation that helped define the system's boundary.

The alerting mechanism also benefited from iteration. Since the project is concerned with resilience, it was not enough to generate alerts only when a remote endpoint was available. The system therefore included persistent queueing so that failed HTTP alerts could be retained and retried after endpoint recovery. This design choice was evaluated through a simulated endpoint outage and recovery experiment.

Figure 3.1 should therefore be interpreted as the development lifecycle of Edge Sentinel. The feedback loop from system evaluation to refinement and correction shows that implementation results were used to improve the design and to bound the final claims of the project.

Figure 3.1: Iterative Prototype-Based Development Model for Edge Sentinel



The diagram should show the following sequence:

Stage	Description	Output
Problem Identification	Identification of weaknesses in passive and cloud-dependent surveillance systems under infrastructure constraints	Defined research problem
Requirements Analysis	Identification of functional, non-functional, hardware, software, and deployment requirements	System requirements
Literature Review and Technology Study	Review of intelligent surveillance, object detection, tracking, edge AI, low-light vision, anomaly detection, and Nigerian deployment challenges	Technical foundation
System Architecture Design	Design of the modular edge surveillance architecture	Initial system architecture
Prototype Implementation	Implementation of capture, detection, tracking, indicators, alerts, queueing, and local interface modules	Working prototype modules
Hardware and Sensor Integration	Integration and verification of camera, PIR sensor, and GSM/SMS modem	Verified hardware components
Module Testing	Testing of individual software and hardware components	Corrected modules
System Evaluation	Scenario evaluation, detector-tracker screening, resource profiling, and alert recovery testing	Evaluation evidence
Refinement and Correction	Adjustment of design and	Improved prototype and

Stage	Description	Output
	claims based on evaluation findings	bounded conclusions
Final Prototype and Documentation	Preparation of final implemented system, results, report, and presentation	Final project deliverables

The feedback loop in the model connects system evaluation and refinement back to system architecture design and prototype implementation. This represents the fact that the project evolved through repeated testing rather than a single one-way development process.

3.4 Requirements Analysis

The requirements analysis defines what the Edge Sentinel prototype was expected to do and the conditions under which it was expected to operate. The requirements were derived from the project problem, literature review, deployment constraints, and implementation goals. Since the system targets infrastructure-constrained environments, the requirements include not only detection and alerting functions, but also local operation, resilience, configurability, and bounded resource usage.

The functional requirements describe the services that the system must provide. These include video capture, person detection, anonymous tracking, behavioural-event reasoning, zone configuration, event logging, evidence handling, alert delivery, queue recovery, and local user interaction. The non-functional requirements describe qualities expected from the system, such as modularity, explainability, resilience, edge suitability, maintainability, and privacy-conscious operation.

Table 3.1: Functional Requirements

Requirement ID	Functional Requirement	Description	Implementation Status
FR1	Video capture	The system shall capture visual input from a camera connected to the edge device.	Implemented and camera capture verified
FR2	Local person detection	The system shall detect persons locally without	Implemented using EfficientDet-Lite0 as selected reference

Requirement ID	Functional Requirement	Description	Implementation Status
		requiring cloud inference.	detector
FR3	Anonymous tracking	The system shall maintain temporary track continuity for detected persons without identifying individuals.	Implemented using ByteTrack
FR4	Zone configuration	The system shall allow monitored areas or zones to be configured for rule-based event evaluation.	Implemented through local configuration workflow
FR5	Restricted presence detection	The system shall generate an event when a person is detected in a restricted or monitored zone.	Implemented and evaluated
FR6	Lingering detection	The system shall generate an event when a tracked person remains in a zone beyond a configured duration.	Implemented and evaluated
FR7	Crowd-density monitoring	The system shall generate an event when the number of persons in a zone exceeds a configured threshold.	Implemented and evaluated
FR8	Tailgating detection	The system shall generate an event	Implemented and evaluated within

Requirement ID	Functional Requirement	Description	Implementation Status
		when movement patterns indicate unauthorized close-following or entry behaviour based on configured rules.	scenario scope
FR9	Abnormal-motion detection	The system shall generate an event when movement behaviour satisfies configured abnormal-motion criteria.	Implemented and evaluated with one known false negative
FR10	Event logging	The system shall store generated event records locally.	Implemented
FR11	Evidence handling	The system shall support event evidence such as snapshots or related event metadata where applicable.	Implemented within prototype scope
FR12	HTTP alert delivery	The system shall attempt to deliver event alerts to a configured HTTP endpoint.	Implemented and evaluated
FR13	Persistent alert queueing	The system shall retain failed alerts and retry delivery after endpoint recovery.	Implemented and evaluated
FR14	GSM/SMS	The system shall support SMS alert	Functionally verified

Requirement ID	Functional Requirement	Description	Implementation Status
	capability	notification through an AT-command modem.	
FR15	PIR-assisted activation	The system shall support motion-sensor input as part of deployment-aware activation.	Functionally verified
FR16	Local configuration interface	The system shall provide a local interface for setup, configuration, and review of system state.	Implemented within prototype scope

Table 3.2: Non-Functional Requirements

Requirement ID	Non-Functional Requirement	Description	Design Response
NFR1	Edge operation	The system should perform core detection and event reasoning locally on Raspberry Pi-class hardware.	Local inference and rule processing are performed on the edge device
NFR2	Resilience	The system should preserve event records and alerts when immediate remote delivery fails.	Persistent alert queueing and retry mechanism are used
NFR3	Modularity	The system should allow major components to be	Detector, tracker, indicators, alerting, and configuration

Requirement ID	Non-Functional Requirement	Description	Design Response
		changed or improved without redesigning the entire system.	logic are separated into modules
NFR4	Explainability	Security events should include understandable trigger reasons.	Rule-based indicators are used instead of opaque anomaly scores
NFR5	Privacy-conscious operation	The system should avoid identity recognition during routine operation.	Anonymous track IDs are used for continuity instead of facial recognition
NFR6	Configurability	The system should support site-specific zones, thresholds, and indicator settings.	Local zone and indicator configuration are supported
NFR7	Resource awareness	The system should be suitable for constrained hardware and should avoid unnecessary heavy computation.	Lightweight detection, ByteTrack tracking, saved-video profiling, and modular scheduling are used
NFR8	Maintainability	The system should be understandable and extendable for future work.	Modular architecture and documented components support maintenance
NFR9	Deployment relevance	The system should address weak internet, limited technical support, and unreliable infrastructure.	Local processing, event logging, queue recovery, and SMS capability support constrained deployment
NFR10	Bounded security	The system should	Evaluation results

Requirement ID	Non-Functional Requirement	Description	Design Response
	claims	not claim capabilities that were not evaluated.	and limitations are reported with claim boundaries

Table 3.3: Hardware and Software Requirements

Category	Requirement	Purpose	Status
Hardware	Raspberry Pi 5	Main edge computing platform for local inference and event reasoning	Used as deployment platform
Hardware	IMX219 NoIR CSI camera	Visual input for surveillance monitoring	Capture verified
Hardware	HC-SR501 PIR sensor	Motion transition input for deployment-aware activation	GPIO17 transition verified
Hardware	SIMCOM A7670G modem	GSM/SMS alert capability	SMS delivery functionally verified
Hardware	Storage medium	Local logs, configuration, queue, and evidence storage	Used within prototype
Hardware	Optional IR illumination	Support for improved low-light visibility	Considered as deployment support, not fully validated
Software	Python runtime and project environment	Main implementation environment	Used for system modules
Software	EfficientDet-Lite0	Selected reference detector for person detection	Used in final reference pipeline

Category	Requirement	Purpose	Status
Software	ByteTrack	Anonymous multi-object tracking	Used in final reference pipeline
Software	Local web/API interface	Configuration, calibration, and review of system state	Implemented within prototype scope
Software	Alert manager and queue	HTTP alert delivery, retry, and recovery	Implemented and evaluated
Software	AT-command SMS client	Communication with GSM modem for SMS capability	Functionally verified
Software	Configuration files	Zone and indicator settings	Used for local configurability
Software	Evaluation scripts and logs	Scenario evaluation, profiling, and alert recovery evidence	Used for project evaluation

The requirements show that Edge Sentinel was designed as more than a detector running on a camera feed. The system combines local perception, anonymous tracking, configurable reasoning, persistent alert handling, and deployment-aware hardware integration. These requirements provide the foundation for the system architecture presented in the next section.

3.5 Overall System Architecture

The overall architecture of Edge Sentinel was designed around the idea that the most important surveillance functions should remain local to the edge device. The system does not treat the camera as a passive recording tool, and it does not depend on continuous cloud connectivity before useful event reasoning can occur. Instead, it uses the edge device as the centre of the surveillance pipeline. The device receives visual input, performs local person detection, maintains anonymous tracking, evaluates configured behavioural rules, stores event records, and attempts alert delivery through available communication channels.

The architecture is organized into seven main layers. These are the sensing layer, perception layer, configuration layer, behavioural reasoning layer, event and evidence layer, communication and resilience layer, and local interface layer. This layered structure was

adopted to keep the system modular. Each layer performs a clear role, while still contributing to the overall purpose of resilient surveillance in infrastructure-constrained environments.

The sensing layer provides input to the system. It consists mainly of the IMX219 NoIR CSI camera and the HC-SR501 PIR sensor. The camera supplies visual frames for detection and tracking, while the PIR sensor provides motion transition input that can support activation or deployment-aware operation. The camera is the main source of visual evidence, while the PIR sensor supports the project's power-conscious and event-aware direction.

The perception layer processes the visual input. It uses EfficientDet-Lite0 for person detection and ByteTrack for anonymous multi-object tracking. The detector identifies persons in the frame, while the tracker links detections across frames using temporary track identifiers. These track identifiers are not linked to human identity. They only provide continuity so that the system can reason about duration, movement, and zone transitions.

The configuration layer stores site-specific information required for event reasoning. This includes monitored zones, restricted areas, rule settings, thresholds, durations, and indicator options. Surveillance environments differ from one another, so the system cannot depend on fixed hard-coded assumptions. A zone that is restricted in one location may be normal in another. A crowd threshold that is acceptable in a hallway may be unacceptable near a laboratory door. For this reason, configurability is part of the system architecture rather than an afterthought.

The behavioural reasoning layer converts detections and tracks into meaningful security events. This layer evaluates rules for restricted presence, lingering, crowd density, tailgating, and abnormal motion. The rules are designed to be explainable. An alert is not generated simply because a black-box model produces an anomaly score. It is generated because a visible condition is satisfied, such as a person entering a restricted zone, staying too long in a monitored area, exceeding a crowd threshold, following too closely through an entry path, or satisfying configured motion criteria.

The event and evidence layer records what happened when a rule is triggered. An event record may contain the event type, timestamp, zone, track information, measured values, configured thresholds, and trigger reason. Evidence such as snapshots or related event metadata can also be attached where applicable. This layer is important because surveillance alerts should not only notify the user. They should also preserve enough information for later review.

The communication and resilience layer handles alert delivery. Under normal conditions, the system attempts to deliver alerts through a configured HTTP endpoint. When delivery fails,

the alert is not discarded. It is placed in a persistent local queue and retried after the endpoint becomes available again. The system also includes GSM/SMS capability through an AT-command modem. In the current evaluation, HTTP queue recovery was formally tested, while SMS delivery was functionally verified separately.

The local interface layer supports configuration, calibration, event review, and system visibility. This makes the system usable during setup and demonstration because users need to configure zones, inspect system state, review generated events, and observe alert queue behaviour. The local interface also reinforces the edge-first nature of the system because setup and review can be performed close to the device.

Figure 3.2: Overall Architecture of Edge Sentinel

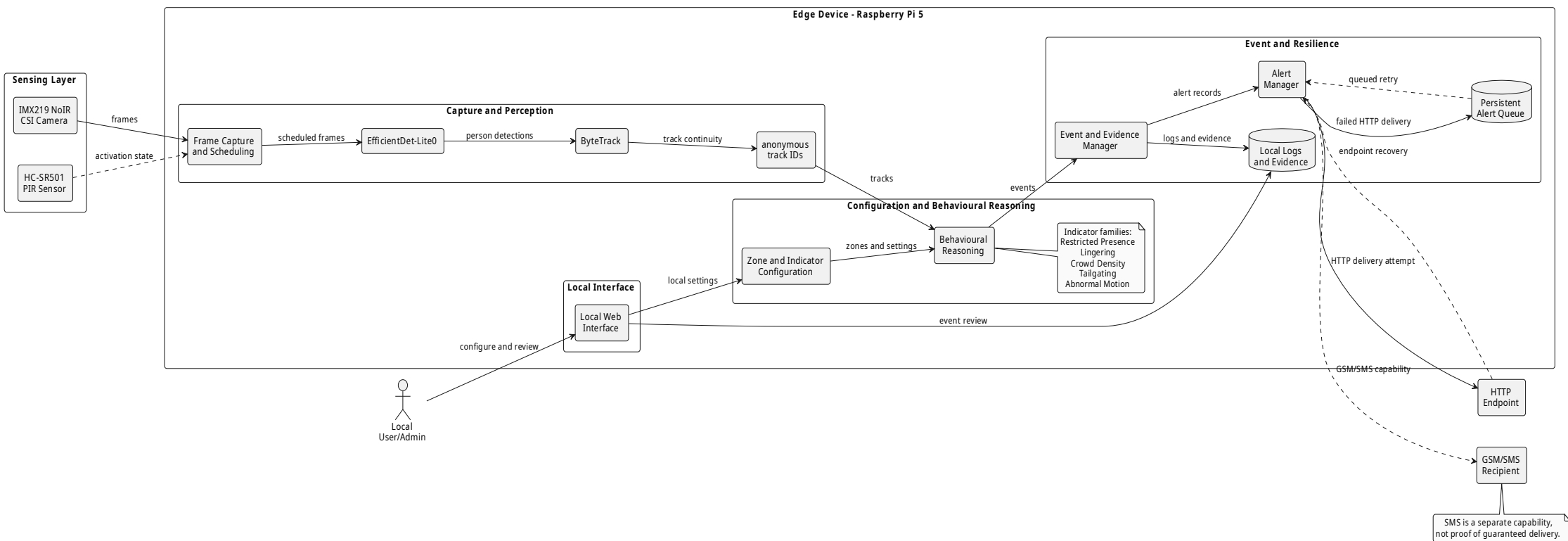


Table 3.4: Overall Architecture Layers

Layer	Main Components	Function in the System
Sensing layer	IMX219 NoIR CSI camera, HC-SR501 PIR sensor	Provides visual input and motion transition input
Perception layer	EfficientDet-Lite0, ByteTrack	Performs local person detection and anonymous tracking
Configuration layer	Zone settings, indicator settings, thresholds, durations	Stores site-specific rules used for event reasoning
Behavioural reasoning layer	Restricted presence, lingering, crowd density, tailgating, abnormal motion indicators	Converts detections and tracks into explainable security events
Event and evidence layer	Event records, snapshots, metadata, local logs	Preserves information about triggered security events
Communication and resilience layer	Alert manager, persistent queue, HTTP client, SMS client	Delivers alerts and retains failed alerts for recovery
Local interface layer	Web dashboard, setup interface, configuration pages, event review	Allows users to configure and inspect the system locally

The layered architecture supports the central aim of the project. Edge Sentinel is not only concerned with detecting persons. It is concerned with maintaining useful surveillance behaviour when infrastructure conditions are imperfect. The architecture therefore combines perception, reasoning, evidence, communication, and recovery into one edge-based prototype.

3.6 Hardware Design

The hardware design of Edge Sentinel was based on the need to build a low-cost and locally deployable surveillance prototype. The selected hardware had to support camera input, local processing, motion sensing, alert communication, and field-oriented experimentation. The hardware design therefore centres on the Raspberry Pi 5 as the edge computing device, with

the IMX219 NoIR CSI camera, HC-SR501 PIR sensor, and SIMCOM A7670G modem serving as supporting components.

The Raspberry Pi 5 acts as the main edge device. It hosts the software pipeline, processes camera frames, runs detection and tracking, evaluates behavioural indicators, stores local logs, manages alert queueing, and serves the local interface. The choice of Raspberry Pi-class hardware reflects the project's focus on affordability and infrastructure-constrained deployment. Although the Raspberry Pi does not provide the same compute capacity as GPU servers or high-end edge accelerators, it is accessible, widely documented, and suitable for demonstrating practical edge surveillance.

The IMX219 NoIR CSI camera provides the visual input for the surveillance pipeline. The NoIR camera was chosen because security monitoring often takes place in low-light or night-time environments, and the absence of an infrared filter allows the camera to work with infrared illumination. In this project, camera capture was verified as part of hardware bring-up. However, the use of a NoIR camera does not mean that pitch-dark surveillance was fully validated. Effective night operation still depends on adequate illumination, camera placement, scene conditions, and later low-light evaluation.

The HC-SR501 PIR sensor provides motion transition input. The PIR sensor is useful because it can detect changes in infrared radiation associated with movement. In the Edge Sentinel design, the PIR sensor supports deployment-aware activation by helping the system react to motion rather than treating all moments equally. This aligns with the project's power-conscious direction, although the project does not claim measured energy savings or complete low-power autonomy. The PIR transition was functionally verified on GPIO17.

The SIMCOM A7670G modem provides GSM/SMS alert capability. This component is important because internet access may not always be stable in the target deployment context. A GSM/SMS fallback path gives the system a way to deliver simple notifications even where normal network-based alerting is unavailable or unreliable. In this project, SMS delivery was functionally verified through the modem using `/dev/ttyUSB2` at 115200 baud. This verification confirms that the system can send SMS messages through the modem, but it does not represent a formal reliability or latency study of SMS delivery.

Local storage is also part of the hardware design. The device requires storage for configuration files, event records, alert queue data, snapshots, logs, and evaluation artifacts. Local storage is important for resilience because alerts and evidence should not disappear simply because a remote endpoint is unavailable at the moment an event occurs.

Optional deployment support such as infrared illumination, solar power, battery backup, and charge control remains relevant to the broader deployment vision. However, these components must be described carefully. The project was motivated by power and lighting constraints, and it explored hardware directions that support such environments, but it does not claim fully measured solar autonomy, watt-level power consumption, or complete pitch-dark surveillance performance.

Figure 3.3: Hardware Architecture of Edge Sentinel

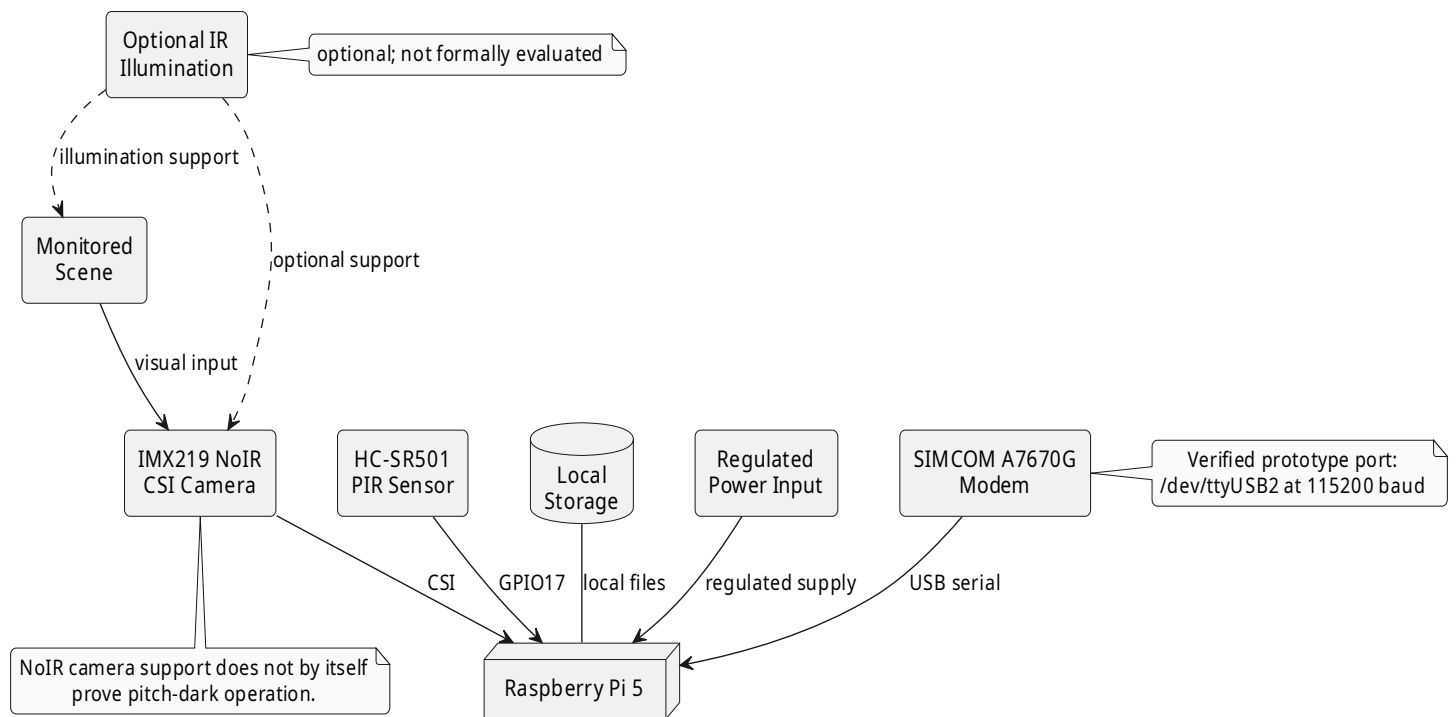


Table 3.5: Hardware Components and Functions

Hardware Component	Function	Verification or Status
Raspberry Pi 5	Main edge computing platform for local inference, tracking, rule evaluation, logging, queueing, and interface hosting	Used as the prototype deployment platform
IMX219 NoIR CSI camera	Captures visual frames for surveillance processing	Camera capture verified
HC-SR501 PIR sensor	Provides motion transition	Transition verified on

Hardware Component	Function	Verification or Status
	input for deployment-aware activation	GPIO17
SIMCOM A7670G modem	Provides GSM/SMS alert capability	SMS delivery functionally verified through /dev/ttyUSB2 at 115200 baud
Local storage	Stores configuration, event records, logs, queue data, and evidence	Used within the prototype
Optional IR illumination	Supports possible low-light or night-time visibility	Considered for deployment support, not fully evaluated
Optional solar and battery system	Supports possible field deployment under unstable power conditions	Deployment direction, not formally validated in the current evaluation

The hardware design reflects the balance between ambition and practicality. Edge Sentinel is intended to show that useful surveillance intelligence can be placed close to the camera using affordable components. At the same time, the project avoids claiming that every possible deployment constraint has been solved. The verified hardware components support the implemented prototype, while some deployment extensions remain future work.

3.7 Software Architecture

The software architecture of Edge Sentinel follows a modular structure. This was necessary because the system combines different types of functionality. It includes camera input, frame scheduling, detection, tracking, rule-based reasoning, event generation, local storage, alert delivery, queue recovery, SMS communication, and a local interface. A monolithic design would make the system difficult to test and extend. The modular design allows individual components to be improved or replaced without changing the entire system.

The capture and scheduling module receives frames from the camera or from saved-video input during evaluation. It controls how frames enter the processing pipeline and helps manage the workload on the edge device. This is important because Raspberry Pi-class hardware cannot be treated like an unlimited compute environment. The system must decide how often detection should run and how frames should be processed so that event reasoning remains responsive without overwhelming the device.

The detector module performs local person detection. In the final reference pipeline, EfficientDet-Lite0 is used as the selected detector. The detector produces bounding boxes and confidence values for persons in the frame. These detections are then passed to the tracking module.

The tracking module uses ByteTrack to maintain anonymous continuity across frames. The tracker assigns temporary track IDs to detections. These IDs allow the system to observe whether the same detected person remains in a zone, moves between zones, or appears to satisfy a motion-based rule. The system does not use these IDs for identity recognition. They are internal labels used only for event reasoning.

The zone and configuration modules provide the context needed to interpret the scene. A detection box alone does not indicate whether a person is in a sensitive area. The zone configuration defines which parts of the scene are monitored and what rules apply to them. The indicator configuration defines thresholds such as linger duration, crowd count, and movement conditions. These configurations allow the same system to be adapted to different environments.

The indicator modules evaluate the configured rules. Each indicator receives relevant detection, tracking, and zone information, then decides whether an event should be generated. Restricted presence depends on zone overlap. Linger depends on duration. Crowd density depends on count. Tailgating depends on transition or close-following behaviour. Abnormal motion depends on movement-related criteria over time. This rule-based design keeps the system explainable and easier to debug.

The event module creates structured event records when indicators are triggered. Each event record can include the event type, timestamp, zone, track ID, measured value, configured threshold, and trigger reason. This structure is important because it allows events to be reviewed later and helps users understand why an alert was generated.

The evidence and logging modules preserve local records. Evidence may include snapshots or event-related metadata. Logs and event records support review, debugging, and evaluation. In infrastructure-constrained environments, local storage is important because remote delivery may fail temporarily. The system should not lose its record of what happened simply because a network endpoint was unavailable.

The alert manager handles outgoing alerts. When an event occurs, the alert manager attempts delivery through a configured HTTP endpoint. If the delivery fails, the alert is written into a persistent queue. The queue is later retried after recovery. This design supports resilience because local event creation can continue during a remote endpoint outage. The SMS module

provides a separate GSM/SMS capability through the AT-command modem, although formal alert recovery evaluation focused on HTTP queueing.

The local API and dashboard provide user access to configuration and system state. This includes setup, zone calibration, indicator settings, event review, and alert queue visibility within the prototype scope. The local interface is important because a surveillance system must be configurable without requiring direct code changes for every deployment.

Figure 3.4: Edge Device Internal Processing Pipeline

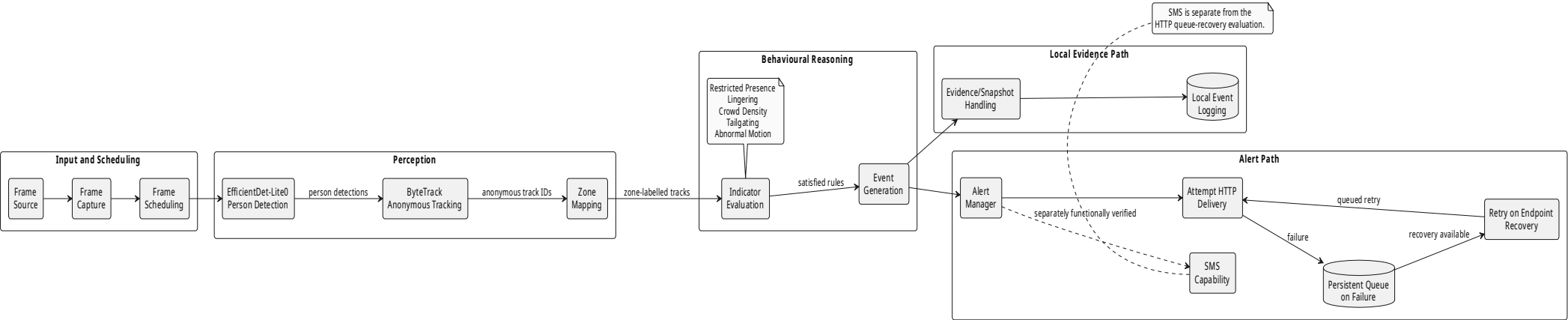


Table 3.6: Main Software Modules

Software Module	Main Function	Role in the System
Capture and scheduling module	Receives frames and controls processing flow	Supplies frames to the detector while managing device workload
Detector module	Performs local person detection	Produces person bounding boxes for tracking and event reasoning
Tracker module	Maintains anonymous track continuity	Supports duration, movement, and transition-based indicators
Zone manager	Stores and applies configured monitoring zones	Provides spatial context for event reasoning
Indicator modules	Evaluate behavioural rules	Generate events for restricted presence, lingering, crowd density, tailgating, and abnormal motion
Event module	Creates structured event records	Records event type, time, zone, track information, and trigger reason
Evidence and logging module	Stores snapshots, metadata, logs, and event records	Preserves local evidence and supports later review
Alert manager	Delivers alerts and manages delivery failures	Sends HTTP alerts and hands failed alerts to the persistent queue
Persistent queue	Stores failed alerts for later retry	Supports alert retention and recovery after endpoint outage
SMS module	Sends messages through the GSM modem	Provides functionally verified fallback notification capability
Local API and dashboard	Provides setup,	Allows user interaction with

Software Module	Main Function	Role in the System
	configuration, and review interface	the local system
Configuration storage	Stores zones, thresholds, and indicator settings	Makes the system configurable across deployment sites

The software architecture supports the modularity objective of the project. Each module has a specific responsibility, and the modules are connected through a processing pipeline. This makes the system easier to evaluate because the behaviour of the detector, tracker, indicators, alerts, and queue can be examined separately before discussing the complete prototype.

3.8 Detection and Tracking Design

Detection and tracking form the perception foundation of Edge Sentinel. The detector identifies persons in individual frames, while the tracker links detections across frames. This combination is necessary because the behavioural indicators depend not only on whether a person is visible, but also on how that person behaves over time.

The detection task in this project focuses on persons. This is because the implemented security indicators are person-centred. Restricted presence, lingering, crowd density, tailgating, and abnormal motion all require the system to know where people are in the scene. The system does not attempt to detect every possible object category or classify detailed human activities. This narrower focus helps keep the prototype practical for edge deployment.

EfficientDet-Lite0 was selected as the reference detector for the final evaluation pipeline. This choice reflects the project's emphasis on lightweight inference and edge suitability. Larger detectors may offer stronger performance in some conditions, but they also increase computation cost and may reduce practical throughput on Raspberry Pi-class hardware. The final selection therefore considered not only detection ability, but also timing behaviour, pipeline stability, and usefulness for the implemented rules.

Tracking was added because detection alone was not enough for the intended behavioural indicators. Without tracking, the system could detect that a person is present in a frame, but it would struggle to know whether the same person remained in a zone for a long time or moved through a sequence of areas. ByteTrack was selected because it provides multi-object tracking based on detection outputs and can maintain temporary track continuity without requiring appearance-based identity recognition.

DeepSORT was not used as the final tracker because it introduces appearance-based re-identification features that are heavier and less aligned with the project’s privacy-conscious and resource-aware direction. Edge Sentinel does not need to recognize who a person is. It only needs to know that a detected person is likely the same track across nearby frames. ByteTrack therefore provides a better fit for the prototype because it supports anonymous continuity while avoiding identity recognition.

The detector-tracker pair used in the selected reference pipeline is EfficientDet-Lite0 with ByteTrack. This pair supports the five behavioural indicators implemented in the system. Restricted presence can be evaluated from the location of detected persons. Lingering can be evaluated from track duration in a zone. Crowd density can be evaluated from the count of persons in a monitored area. Tailgating can be evaluated from transition patterns or close-following behaviour. Abnormal motion can be evaluated from movement-related track information.

YOLOv8n with ByteTrack was later tested as a sensitivity analysis for the difficult running-past case. The purpose of that experiment was not to reopen model selection. It was used to examine whether a different detector would change the behaviour of the abnormal-motion edge case. The result showed stronger instantaneous speed evidence, but the scenario still failed the sustained-duration requirement. YOLOv8n therefore did not replace EfficientDet-Lite0 in the final reference pipeline.

The detection and tracking design is evaluated in this project using practical system-level evidence rather than formal detector and tracker benchmark metrics. The project does not claim detector mAP, detector accuracy, MOTA, IDF1, or HOTA. Instead, it evaluates whether the selected detector-tracker pair supports the implemented surveillance rules under the tested scenarios, and whether the pipeline can run with acceptable stability and timing on the Raspberry Pi.

Table 3.7: Detector-Tracker Design Considerations

Design Consideration	Reason for Consideration	Final Design Response
Person-focused detection	The implemented behavioural indicators depend mainly on human presence and movement	The detector is used primarily for person detection
Lightweight inference	The system targets Raspberry Pi-class edge deployment	EfficientDet-Lite0 was selected as the reference detector

Design Consideration	Reason for Consideration	Final Design Response
Temporal continuity	Lingering, tailgating, and abnormal motion require continuity across frames	ByteTrack was used for anonymous multi-object tracking
Privacy-conscious operation	The system should avoid identity recognition during routine operation	Temporary track IDs are used instead of facial identity or re-identification
Explainable event reasoning	Security events should be traceable to understandable conditions	Rule-based indicators use detection, tracking, zone, count, duration, and movement data
Practical edge performance	The perception pipeline must support local processing without excessive delay	Detector-tracker selection considered timing behaviour and system stability
Bounded evaluation	The project should not claim metrics that were not measured	Results are reported using scenario outcomes, profiling, and alert recovery evidence rather than mAP or MOTA

The detection and tracking design supports the broader purpose of Edge Sentinel. The system does not attempt to solve all surveillance intelligence problems through one large model. Instead, it uses a lightweight detector and anonymous tracker to provide enough perception for practical, explainable, rule-based surveillance events. This makes the system more suitable for the deployment constraints addressed in the project.

3.9 Behavioural Indicator Design

The behavioural indicator layer is the part of Edge Sentinel that converts detection and tracking outputs into security events. The system does not attempt to classify every possible human activity. Instead, it focuses on a set of practical surveillance indicators that can be expressed through visible and explainable conditions. This approach was selected because the project targets edge deployment, where computation, memory, power, and reliability must be considered alongside detection performance.

The behavioural indicators use information from the detector, tracker, zone configuration, and indicator settings. The detector provides person locations. The tracker provides

temporary continuity across frames. The zone configuration defines the parts of the scene that are important for monitoring. The indicator settings define thresholds such as duration, count, transition conditions, or movement criteria. By combining these sources, the system can reason about selected behaviours without using identity recognition or a heavy general-purpose activity recognition model.

The first indicator is restricted presence or restricted access. This indicator checks whether a detected and tracked person is present inside a configured restricted zone. A restricted zone may represent a laboratory entrance, a staff-only area, a storage section, or any monitored region defined during setup. When a person overlaps that zone and the rule is active, the system can generate a restricted-presence event. The purpose of this rule is to support awareness of unauthorized or sensitive-area presence.

The second indicator is lingering. Lingering occurs when a tracked person remains within a monitored area beyond a configured duration. This indicator requires tracking because it is not enough to know that a person appeared in one frame. The system must estimate whether the same tracked person has remained in the area across time. Lingering is useful for detecting suspicious waiting, loitering near entrances, or unusual presence around sensitive areas.

The third indicator is crowd density. Crowd density is based on the number of persons detected within a configured zone or scene area. When the count exceeds a defined threshold, the system can generate a crowd-density event. This indicator is useful in areas where abnormal crowding may suggest congestion, unauthorized gathering, or safety risk. Since the prototype uses person detections and zone context, the crowd-density rule remains explainable and configurable.

The fourth indicator is tailgating. Tailgating refers to a situation where one person closely follows another through an entry path or controlled region. In the system design, tailgating depends on tracking and zone transitions. The system must observe movement through configured areas and evaluate whether the pattern satisfies the rule condition. This indicator is more complex than simple restricted presence because it depends on temporal relationships between tracked persons and configured zones.

The fifth indicator is abnormal motion. Abnormal motion is used to identify movement behaviour that satisfies configured motion criteria, such as unusually fast movement across a scene or monitored area. This indicator depends on track displacement and timing information. Unlike a full deep-learning-based anomaly detection model, the abnormal-motion rule is based on measurable movement conditions. This makes the rule easier to

interpret, although it also means that the system may miss short or poorly sustained motion events. The known limitation of the running-past case is discussed later in Chapter Four.

The use of rule-based behavioural indicators makes Edge Sentinel easier to configure and defend. Each alert can be connected to a clear condition. A user can understand that an event occurred because a person entered a restricted zone, stayed too long, exceeded a crowd threshold, moved through an entry pattern, or satisfied a motion rule. This is important in security systems because alerts should not only be generated. They should also be understandable.

Table 3.8: Behavioural Indicators and Rule Logic

Indicator	Main Input Used	Rule Condition	Output Event	Evaluation Status
Restricted presence/access	Detection box, track ID, configured zone	A tracked person overlaps a restricted or monitored zone while the rule is active	Restricted-presence or restricted-access event	Implemented and evaluated
Lingering	Track ID, zone overlap, duration setting	A tracked person remains in a monitored zone longer than the configured duration	Lingering event	Implemented and evaluated
Crowd density	Person detections, zone overlap, count threshold	The number of detected persons in a monitored area exceeds the configured maximum	Crowd-density event	Implemented and evaluated
Tailgating	Track IDs, zone transitions, timing or proximity	Movement through configured areas satisfies	Tailgating event	Implemented and evaluated within scenario scope

Indicator	Main Input Used	Rule Condition	Output Event	Evaluation Status
	condition	the tailgating rule condition		
Abnormal motion	Track displacement, timing, movement threshold	A tracked person satisfies configured abnormal-motion criteria	Abnormal-motion event	Implemented and evaluated with a known short-crossing limitation

The behavioural indicator design supports the central balance in the project. The system is intelligent enough to reason about selected security conditions, but simple enough to run on constrained hardware and explain its decisions. The indicators are therefore a practical middle ground between passive CCTV and heavy black-box video anomaly detection.

3.10 Zone Configuration and Local Setup Interface

Zone configuration is essential to the operation of Edge Sentinel because surveillance meaning depends on the environment being monitored. A person standing in one part of a scene may be normal, while the same person standing in another part may be suspicious or unauthorized. For this reason, the system must allow monitored areas and rule settings to be configured according to the deployment site.

A zone is a defined region in the camera view. It may represent a restricted area, an entrance path, a doorway region, a corridor section, or any part of the scene that the user wants to monitor. Behavioural indicators use these zones to determine whether an event should be triggered. Without zones, the system would only know that people are present in the frame. With zones, the system can reason about where the person is, how long the person has remained there, and whether movement through the scene has security meaning.

The local setup interface supports this configuration process. It provides a way to capture or view a camera frame, define monitoring zones, assign indicator settings, and inspect relevant system state. This is important because the system is intended to be configurable rather than hard-coded for one environment. A surveillance prototype that requires code changes for every new scene would not be practical for deployment.

The zone configuration process also supports explainability. When an event is generated, the user can relate the event to a visible zone and a configured rule. For example, a restricted-presence event can be understood by checking the restricted zone. A lingering event can be understood by reviewing the configured duration threshold. A crowd-density event can be

understood by comparing the observed count with the configured maximum. This makes the system easier to tune and verify.

The local interface also supports event review and alert queue visibility within the prototype scope. These features are useful because resilience is not only about detecting events. It is also about knowing whether events were recorded, whether alerts were delivered, and whether any alerts remain queued after a delivery failure. A local interface gives the user a way to inspect these states without relying on a remote cloud dashboard.

Figure 3.5: Zone-Based Configuration Concept

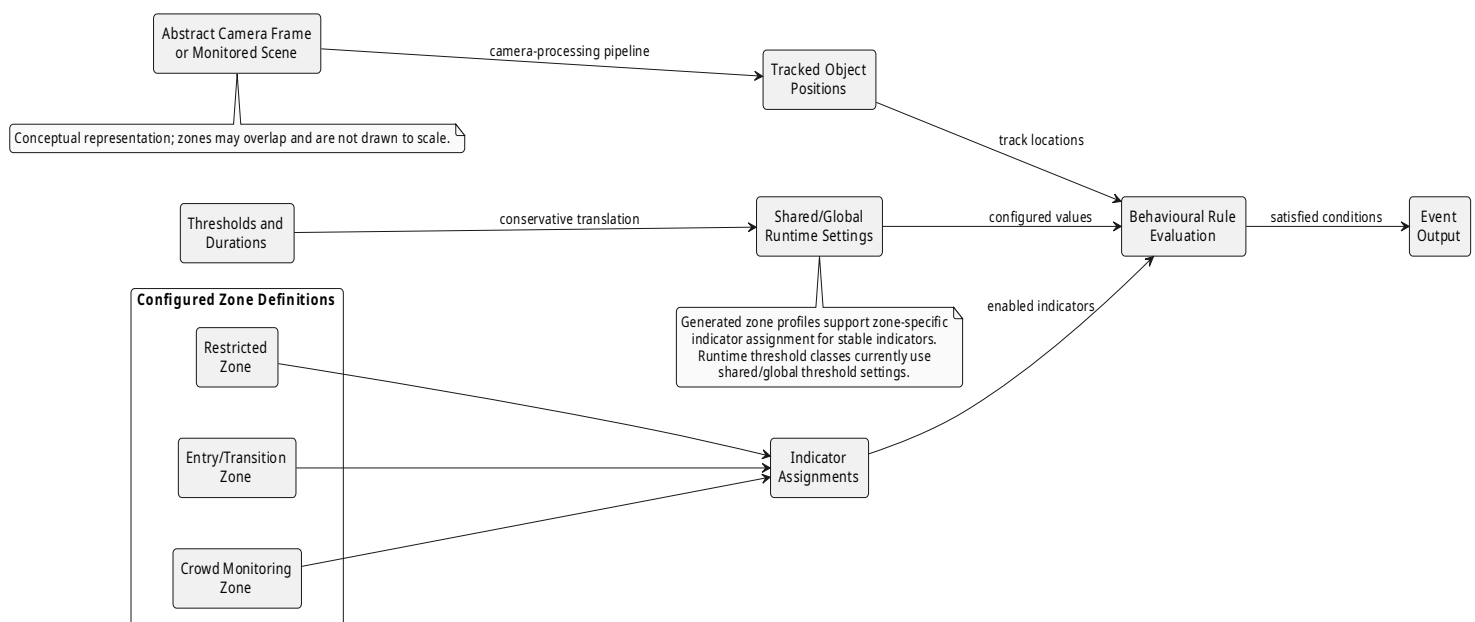


Table 3.9: Zone Configuration Elements

Configuration Element	Purpose	Example Use
Zone name	Identifies the monitored area	Entrance, restricted area, storage corner, laboratory door
Zone coordinates	Defines the region of interest in the camera frame	Rectangle or polygon drawn over a calibration frame
Active indicators	Specifies which rules apply to the zone	Restricted presence, lingering, crowd density
Duration threshold	Defines how long a person may remain before an event is triggered	Lingering after a configured number of seconds
Count threshold	Defines the maximum allowed number of persons in a zone	Crowd-density event when count exceeds the limit
Transition condition	Defines movement relationships across zones	Tailgating or entry-pattern evaluation
Motion threshold	Defines movement criteria for abnormal-motion detection	Fast or unusual movement based on track displacement and time

The zone configuration design makes Edge Sentinel adaptable. The same software can support different monitoring scenes by changing zones and thresholds. This aligns with the project's modularity objective and supports deployment in varied infrastructure-constrained environments.

3.11 Event, Evidence, and Logging Design

The event, evidence, and logging layer is responsible for preserving what the system observes when a behavioural rule is triggered. This layer is important because a surveillance system should not only detect events in memory and then forget them. It should produce records that can be reviewed, explained, and used to support later response or analysis.

An event is created when one of the behavioural indicators satisfies its configured rule condition. The event record contains structured information about the incident. This may include the event type, timestamp, zone name, track identifier, trigger reason, measured

value, configured threshold, and delivery state. The exact fields may vary depending on the event type, but the purpose is the same. The system should record enough information to explain why the event was generated.

The use of structured event records supports both evaluation and real deployment. During evaluation, event records help determine whether the system produced the expected event for each scenario. During deployment, event records allow users to review what happened and when it happened. This also helps debugging because the event record can reveal whether a rule was triggered by zone overlap, duration, count, transition, or movement.

Evidence handling extends the value of the event record. In a visual surveillance system, a text alert alone may not be enough. A snapshot or related visual evidence can help the user verify the situation. Edge Sentinel supports evidence capture within the prototype scope, especially for security events where later review may be required. However, evidence capture must be described as event-based and bounded. The system is not presented as a continuous cloud recording platform.

Local logging is central to the resilience goal of the project. If remote alert delivery fails, the system should still preserve the local record of the event. This means that communication failure should not erase the fact that the event occurred. Local logs and queue files therefore play an important role in maintaining continuity between event detection, alert delivery, and recovery.

The design also supports privacy-conscious operation. The system does not require facial recognition or identity classification for routine event reasoning. It uses anonymous track IDs for temporary continuity. Evidence is associated with triggered security events rather than being treated as identity recognition output. This does not remove all privacy concerns, but it makes the design more limited and responsible than identity-based surveillance.

Table 3.10: Event Record Schema

Field	Description	Purpose
Event ID	Unique identifier for the generated event	Allows event tracking and reference
Event type	Category of event generated	Identifies restricted presence, lingering, crowd density, tailgating, or abnormal motion
Timestamp	Time the event was	Supports review and

Field	Description	Purpose
	generated	ordering of events
Zone name	Name of the zone associated with the event	Provides spatial context
Track ID	Temporary anonymous identifier of the tracked person where applicable	Supports continuity without identity recognition
Trigger reason	Human-readable explanation of why the event was generated	Supports explainability
Measured value	Observed value that satisfied the rule	May represent count, duration, movement, or transition evidence
Configured threshold	Threshold used by the rule	Allows comparison between observed condition and configured setting
Evidence path	Location of snapshot or related evidence where available	Supports visual review
Delivery status	Indicates whether alert delivery succeeded, failed, or was queued	Supports alert recovery and monitoring
Retry metadata	Information about retry attempts where applicable	Supports persistent alert delivery

The event design gives Edge Sentinel its audit trail. It connects perception, reasoning, evidence, and alerting into a traceable record. This is necessary because the value of a surveillance system depends not only on detecting something, but also on preserving enough context to support action and review.

3.12 Alerting and Resilience Design

The alerting and resilience layer is one of the most important parts of Edge Sentinel because the project is motivated by infrastructure-constrained deployment. In many environments, internet connectivity may be unstable or temporarily unavailable. A surveillance system that

loses alerts during communication failure would be unreliable, even if its detection pipeline works correctly. The alerting design therefore focuses on retention and recovery.

When a behavioural event is generated, the alert manager attempts to deliver the alert to a configured HTTP endpoint. This represents the normal network-based alert path. If the endpoint is available, the alert can be delivered immediately. If the endpoint is unavailable or delivery fails, the alert is not discarded. Instead, it is stored in a persistent local queue. The queue can then be retried after the endpoint recovers.

This queue-based design supports graceful degradation. During a remote endpoint outage, the system can continue local detection, event reasoning, event logging, and alert queueing. Remote delivery may be delayed, but local event creation does not stop. Once the endpoint becomes available again, queued alerts can be delivered. This behaviour was evaluated in the project using a simulated HTTP endpoint outage and recovery test.

The alert queue is important because it changes how communication failure is handled. Without a queue, a failed alert may be lost permanently. With a persistent queue, the system preserves the alert until delivery becomes possible. This is especially useful in infrastructure-constrained environments where network conditions may fluctuate. The queue does not solve every possible network problem, but it provides a practical recovery mechanism for delivery failure.

The system also includes GSM/SMS capability through the SIMCOM A7670G modem. SMS is relevant because it can serve as a simple fallback notification path when internet-based delivery is not suitable. In the current project, SMS delivery was functionally verified separately from the HTTP queue recovery evaluation. This means the report can state that SMS capability was verified, but it should not claim guaranteed SMS reliability, SMS latency, or formal SMS recovery performance.

The resilience design must therefore be described with careful boundaries. The project evaluated alert retention and recovery under simulated HTTP endpoint outage. It did not perform a full physical network-disconnection campaign, did not measure internet or cloud recovery latency, and did not validate guaranteed SMS delivery under all mobile network conditions. These limitations do not weaken the design. They make the evaluation honest and define what remains for future work.

Figure 3.6: Alert Delivery and Queue Recovery Flow

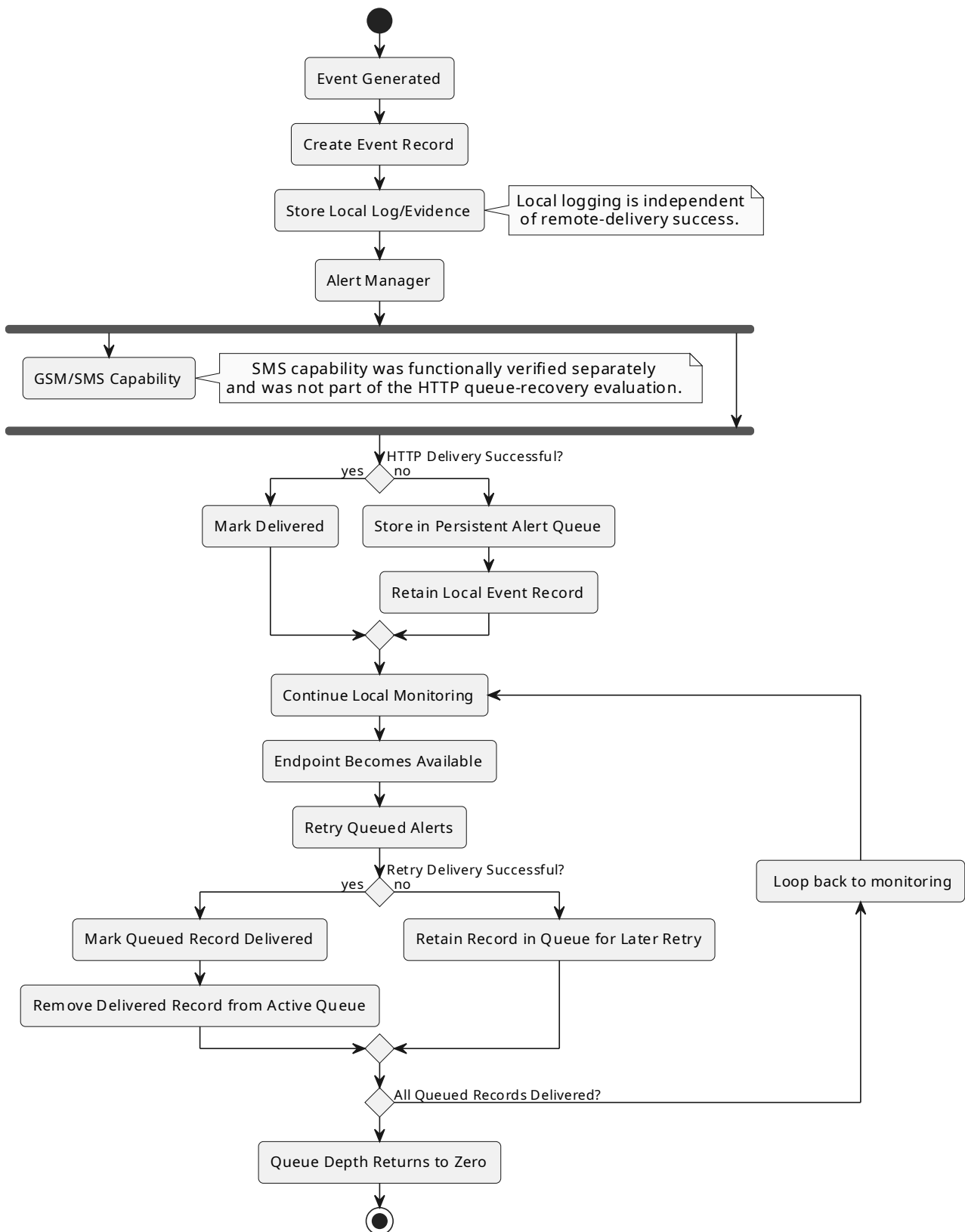


Table 3.11: Alerting and Resilience Design

Component	Function	Resilience Contribution
Alert manager	Receives event alerts and attempts delivery	Centralizes alert handling
HTTP delivery client	Sends alerts to a configured endpoint	Supports normal network-based notification
Persistent queue	Stores failed alerts locally	Prevents alert loss during endpoint outage
Retry mechanism	Attempts delivery of queued alerts after recovery	Supports delayed delivery
Local event log	Stores event records independent of remote delivery	Preserves event history during communication failure
SMS client	Sends SMS messages through the GSM modem	Provides functionally verified fallback notification capability
Local interface	Displays relevant system and queue state within prototype scope	Helps users inspect alert and recovery status

The alerting and resilience design reflects the main contribution of the project. Edge Sentinel is not only designed to detect selected events. It is designed to preserve event information and recover alert delivery when remote communication fails. This makes the system more suitable for environments where reliable connectivity cannot be assumed.

3.13 Operational State Model

The operational state model describes how Edge Sentinel behaves during normal monitoring, event processing, alert delivery, and recovery. This model is important because the system is not a simple script that only runs detection on a video file. It is a surveillance prototype that moves through different states depending on sensor input, frame availability, detection results, rule outcomes, alert delivery status, and shutdown conditions.

The system begins in an idle or ready state. In this state, the device is available for monitoring and can receive input from the camera pipeline or PIR sensor. The PIR sensor supports motion-aware activation, although the system does not claim deep sleep or complete

low-power autonomy. When motion or an active monitoring condition is present, the system moves into an active processing state.

In the active state, frames are captured or read from the selected input source. During evaluation, saved-video workloads were used for repeatable testing. During deployment-oriented operation, the camera supplies the visual input. The frame then enters the processing pipeline, where scheduling determines whether detection should be performed. This scheduling is necessary because the edge device has limited processing capacity and the system must avoid unnecessary overload.

When a frame is selected for inference, the detector identifies persons in the scene. The resulting detections are passed to the tracker, which maintains temporary anonymous track continuity. The tracked outputs are then combined with zone configuration and indicator settings. The behavioural reasoning layer evaluates whether any configured rule has been satisfied.

If no rule is triggered, the system continues monitoring and processing subsequent frames. If a rule is triggered, an event is generated. The event record is stored locally and may be associated with evidence such as a snapshot or relevant metadata. The alert manager then attempts to deliver the alert through the configured HTTP endpoint. If delivery succeeds, the alert is marked as delivered. If delivery fails, the alert is stored in the persistent queue for retry.

The recovery state becomes important when failed alerts exist in the queue. The system can retry queued alerts after the endpoint becomes available again. Once the queued alerts are delivered, the queue depth returns to zero. This recovery behaviour supports the resilience objective of the project because events are not lost simply because immediate remote delivery fails.

The shutdown state represents controlled termination of the system. A clean shutdown is important because logs, queue files, and runtime state should not be corrupted. During resource profiling, the system completed the evaluated saved-video workload and shut down cleanly.

Figure 3.7: Operational State Machine

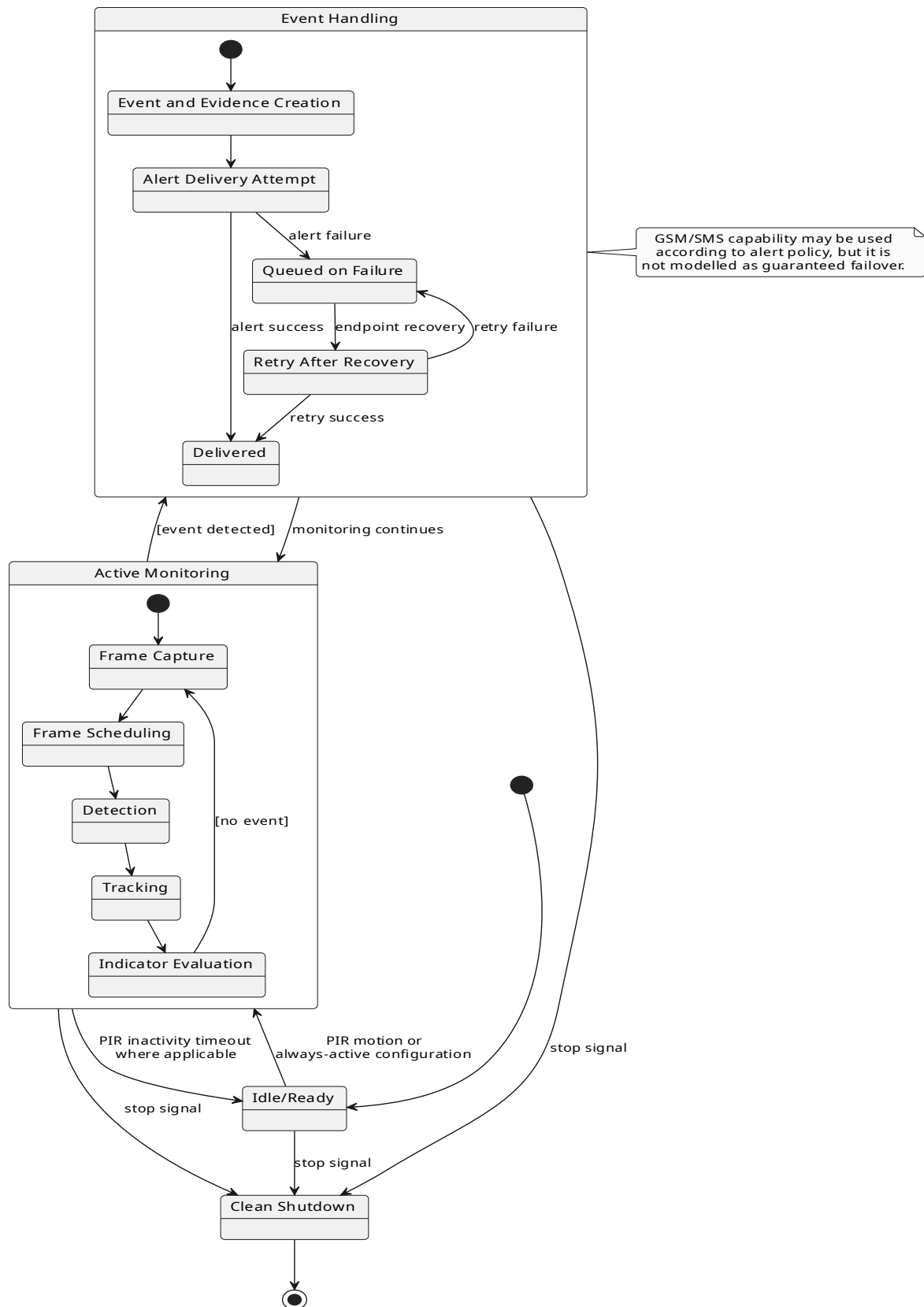


Table 3.12: Operational States of Edge Sentinel

State	Description	Transition Condition
Idle or ready	The system is available for monitoring and waits for active processing conditions	Motion input, active monitoring, or frame availability
Active processing	Frames are captured or read and prepared for analysis	Frame enters scheduling stage
Frame scheduling	The system determines whether detection should be performed on the current frame	Scheduled detector run or skipped detector interval
Detection	EfficientDet-Lite0 performs person detection	Detections are produced
Tracking	ByteTrack maintains anonymous continuity across frames	Tracks are updated
Indicator evaluation	Configured behavioural rules are evaluated using detections, tracks, and zones	Rule passes or no rule passes
Event generation	A structured event is created when a rule condition is satisfied	Event is logged and passed to alert manager
Alert delivery	The system attempts HTTP delivery of the generated alert	Delivery succeeds or fails
Queueing	Failed alerts are stored in the persistent local queue	Endpoint unavailable or delivery error occurs
Recovery retry	Queued alerts are retried after endpoint recovery	Endpoint becomes available
Clean shutdown	Runtime resources are released and the system stops safely	User stop command or workload completion

The operational state model shows how the system supports both event detection and alert resilience. The important point is that local event creation and logging are not dependent on

immediate remote delivery. This separation is what allows the system to continue preserving surveillance events even during communication failure.

3.14 Evaluation Methodology

The evaluation methodology was designed to test the prototype from different angles. Since Edge Sentinel is a complete edge surveillance system, it would not be sufficient to evaluate only the detector or only the alerting module. The system had to be examined in terms of detector-tracker suitability, behavioural event outcomes, resource behaviour on Raspberry Pi hardware, alert retention and recovery, and functional hardware verification.

The first evaluation category was detector-tracker screening. The purpose of this stage was to identify a practical perception pipeline for the implemented behavioural indicators. The selected reference pair was EfficientDet-Lite0 with ByteTrack. The evaluation emphasis was not on formal detector benchmark metrics such as mAP, and it was not on formal tracking metrics such as MOTA, IDF1, or HOTA. Instead, the screening focused on whether the detector-tracker pair could provide useful person detection and anonymous track continuity for the implemented rule-based indicators under the project's hardware constraints.

The second evaluation category was behavioural scenario evaluation. This was used to test whether the implemented indicators produced the expected outcomes for selected surveillance situations. The evaluated indicators included restricted presence, lingering, crowd density, tailgating, and abnormal motion. Each scenario had an expected outcome, and the system output was compared with that expectation. This provided scenario-level evidence of whether the rule-based event reasoning worked as intended.

The third evaluation category was sustained Raspberry Pi resource profiling. This evaluation examined how the selected pipeline behaved during a sustained saved-video workload. The profiling focused on processed frame rate, detector call rate, detector latency, end-to-end processing latency, temperature, dropped or failed frames, throttling, undervoltage, and shutdown behaviour. This evaluation was important because a surveillance system for constrained environments must be assessed not only by event correctness, but also by whether it can run stably on the target edge device.

The fourth evaluation category was alert queue and recovery testing. Since resilience is a central contribution of the project, the alerting system had to be evaluated under delivery failure. The test simulated a remote HTTP endpoint outage, allowed alerts to be generated and queued, then restored the endpoint and observed whether queued alerts were delivered. This evaluation measured alert retention and recovery within the tested conditions. It did not

represent full physical network-disconnection testing, internet latency testing, or SMS reliability testing.

The fifth category was functional hardware verification. This was used to confirm that important hardware components could operate with the prototype. The IMX219 NoIR camera was verified for capture. The HC-SR501 PIR sensor was verified for motion transition on GPIO17. The SIMCOM A7670G modem was verified for SMS delivery through the selected serial interface. These were functional verification tests rather than repeated reliability experiments.

Table 3.13: Evaluation Categories and Purpose

Evaluation Category	Purpose	Evidence Produced
Detector-tracker screening	To identify a practical reference perception pipeline for edge-based event reasoning	Selection of EfficientDet-Lite0 with ByteTrack
Behavioural scenario evaluation	To test whether configured indicators produced expected event outcomes	Scenario-level precision, recall, F1, and false-negative analysis
Sustained Raspberry Pi profiling	To examine processing stability and resource behaviour under saved-video workload	FPS, detector call rate, latency, temperature, throttling, undervoltage, frame failure, and shutdown observations
Alert queue and recovery testing	To test alert retention during endpoint outage and delivery after recovery	Queue retention, recovery delivery, lost-alert count, duplicate count, and final queue depth
Functional hardware verification	To confirm operation of camera, PIR sensor, and GSM/SMS modem	Capture verification, GPIO transition verification, and SMS delivery verification

Table 3.14: Behavioural Scenario Evaluation Design

Evaluation Item	Description
Evaluation target	Rule-based behavioural indicators

Evaluation Item	Description
Indicators covered	Restricted presence, lingering, crowd density, tailgating, and abnormal motion
Input type	Labelled or expected-outcome scenario videos
Output checked	Whether the expected event outcome was produced
Metric type	Scenario-level event outcome metrics
Main metrics	True positives, false positives, false negatives, precision, recall, and F1
Important boundary	Metrics describe scenario-event outcomes, not detector accuracy or tracker benchmark performance

Table 3.15: Resource Profiling Metrics

Metric	Purpose
Processed FPS	Measures the rate at which frames were processed in the evaluated workload
Detector calls per second	Measures how often detector inference was performed
Mean detector latency	Measures the average time spent in detector inference
Mean end-to-end processing latency	Measures average processing latency through the evaluated pipeline
Peak temperature	Indicates thermal behaviour during sustained workload
Throttling status	Indicates whether the device reduced performance due to thermal or power conditions
Undervoltage status	Indicates whether power supply instability was detected
Dropped or failed frames	Indicates whether the workload produced

Metric	Purpose
	frame processing failures
Shutdown behaviour	Indicates whether the system terminated cleanly

Table 3.16: Alert Resilience Test Design

Test Element	Description
Failure condition	Simulated HTTP endpoint outage
Expected system behaviour	Alerts should be created locally and retained in the persistent queue
Recovery condition	HTTP endpoint becomes available again
Expected recovery behaviour	Queued alerts should be delivered after recovery
Main measurements	Number of alerts generated, queued, recovered, lost, duplicated, and remaining in queue
Important boundary	The receiver was local, so recovery timing represents local queue-flushing overhead rather than internet, cloud, WAN, or SMS latency

The evaluation methodology reflects the nature of the project. Edge Sentinel is not evaluated as a dataset-only machine learning model. It is evaluated as an integrated edge surveillance prototype. For this reason, the evaluation combines scenario outcomes, edge-device profiling, alert recovery, and hardware verification.

3.15 Ethical, Privacy, and Scope Considerations

Visual surveillance systems raise ethical and privacy concerns because they observe people and may produce records of human presence or behaviour. For this reason, Edge Sentinel was designed with a privacy-conscious approach. The system does not perform facial recognition, identity classification, or personal identification during routine operation. It uses anonymous track identifiers only to maintain continuity across frames for rule-based event reasoning.

The use of anonymous tracking reduces the need to attach real-world identity to a detected person. A track ID only indicates that a detection in one frame is likely associated with a

detection in another nearby frame. This is enough to support rules such as lingering or movement analysis, but it does not identify who the person is. This design choice is important because the project is focused on behavioural security events rather than personal identity.

The system also uses event-based evidence rather than presenting itself as a complete continuous cloud recording platform. Evidence is associated with triggered security events and is intended to support review and verification. This helps balance security usefulness with responsible data handling. However, privacy risk is not completely removed. Any system that captures images of people must still be deployed lawfully, with appropriate permission, awareness, access control, and data handling practices.

The project also has technical scope boundaries. Edge Sentinel does not claim perfect anomaly detection. It does not claim full activity recognition across all human behaviours. It does not claim detector mAP, formal tracking accuracy, measured power consumption in watts, full solar autonomy, full physical network-failure validation, guaranteed SMS delivery, or pitch-dark surveillance performance. These boundaries are necessary because the evaluation focused on the implemented prototype and the evidence actually collected.

The ethical position of the project is therefore practical and bounded. Edge Sentinel attempts to reduce unnecessary identity processing while still allowing evidence capture for genuine security events. It also attempts to make alerts explainable through rule-based trigger reasons. These choices make the system easier to review, configure, and justify than a black-box identity-based surveillance system.

3.16 Summary of Chapter

This chapter presented the methodology and system design for Edge Sentinel. The project adopted an iterative prototype-based development methodology because the system involved several interacting components, including computer vision, embedded hardware, sensor input, tracking, behavioural-event logic, alert communication, local storage, and a configuration interface. The methodology allowed the system to evolve through repeated design, implementation, testing, evaluation, and refinement.

The chapter also presented the requirements analysis, overall system architecture, hardware design, software architecture, detection and tracking design, behavioural indicator design, zone configuration, event and logging structure, alerting and resilience design, operational state model, and evaluation methodology. The architecture was organized around local edge processing, anonymous tracking, explainable rule-based event reasoning, persistent alert queueing, and bounded fallback communication.

The design shows that Edge Sentinel is more than a simple object detector attached to a camera. It is a modular edge surveillance prototype designed to support local reasoning, event preservation, and alert recovery in infrastructure-constrained environments. The next chapter presents the implementation, results, and discussion of the evaluated prototype.

CHAPTER FOUR

IMPLEMENTATION, RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the implementation, results, and discussion of the Edge Sentinel prototype. The purpose of the chapter is to show how the system described in Chapter Three was implemented and how the completed prototype performed during evaluation. While Chapter Three focused on design, architecture, and methodology, this chapter focuses on what was actually built, tested, observed, and learned.

The implementation was carried out as a modular edge surveillance system running on Raspberry Pi-class hardware. The system combines camera input, local person detection, anonymous tracking, configurable behavioural indicators, event logging, evidence handling, persistent alert queueing, HTTP alert delivery, and GSM/SMS capability. The implemented behavioural indicators include restricted presence, lingering, crowd density, tailgating, and abnormal motion.

The evaluation was designed to reflect the system-level nature of the project. Edge Sentinel was not evaluated only as an object detection model, because the main contribution of the project is not limited to detection accuracy. Instead, the evaluation examined whether the complete prototype could support local event reasoning, stable edge processing, and alert recovery under a communication failure condition. The results therefore cover hardware verification, detector-tracker selection, behavioural scenario outcomes, Raspberry Pi resource profiling, and alert queue recovery.

The chapter also discusses the limitations discovered during evaluation. One scenario, `abnormal_motion__running_past_frame`, produced a false negative because the detected motion did not satisfy the sustained-duration requirement. This result is important because it shows a real boundary of the current rule logic. The chapter also explains why YOLOv8n was tested as a sensitivity experiment for this case, why it did not replace EfficientDet-Lite0, and why the project avoids unsupported claims such as detector mAP, formal tracking accuracy, measured power consumption, or guaranteed SMS reliability.

4.2 Implementation Overview and Experimental Environment

The Edge Sentinel prototype was implemented as an edge-based surveillance system with local processing as the core design principle. The Raspberry Pi 5 served as the main edge device. It hosted the detection and tracking pipeline, behavioural indicator logic, alert manager, persistent queue, configuration files, local logs, and the local setup interface. The system was designed so that event detection and local logging could continue even when immediate remote alert delivery was unavailable.

The visual input component was based on the IMX219 NoIR CSI camera. The camera was used as the main capture device during hardware verification and deployment-oriented testing. For repeatable evaluation and profiling, saved-video workloads were also used. This distinction is important because saved-video profiling provides controlled and repeatable evidence of processing behaviour, but it should not be described as live-camera FPS performance.

The selected reference perception pipeline was EfficientDet-Lite0 with ByteTrack. EfficientDet-Lite0 provided local person detection, while ByteTrack maintained anonymous track continuity across frames. The tracker did not identify people. It only assigned temporary track identifiers so that the system could reason about duration, movement, zone overlap, and transition patterns.

The behavioural reasoning layer used configured rules to generate security events. Each rule used detection, tracking, zone, duration, count, transition, or movement data depending on the event type. This allowed the system to generate explainable events rather than relying on a black-box anomaly score. The event and alerting layers then recorded the event locally and attempted remote delivery through the configured alert path.

The alerting implementation included HTTP delivery and persistent queueing. When HTTP delivery failed, the alert was stored locally and retried after endpoint recovery. GSM/SMS capability was also implemented through a SIMCOM A7670G modem and functionally verified separately. The HTTP queue recovery evaluation and the SMS verification should be interpreted separately because they tested different parts of the communication design.

Table 4.1: Experimental Environment

Component	Description	Role in Evaluation
Edge device	Raspberry Pi 5	Main platform for local inference, tracking, event reasoning, alert handling,

Component	Description	Role in Evaluation
		and profiling
Camera	IMX219 NoIR CSI camera	Visual input device, capture function verified
Motion sensor	HC-SR501 PIR sensor	Motion transition input, verified on GPIO17
GSM modem	SIMCOM A7670G	SMS alert capability, functionally verified through serial communication
Detector	EfficientDet-Lite0	Selected reference detector for person detection
Tracker	ByteTrack	Selected anonymous multi-object tracker
Behavioural indicators	Restricted presence, lingering, crowd density, tailgating, abnormal motion	Rule-based event generation
Alert mechanism	HTTP delivery with persistent queue, GSM/SMS capability	Alert delivery, retention, recovery, and fallback capability
Profiling workload	Saved-video workload	Repeatable measurement of processing performance and device stability
Alert resilience workload	Simulated HTTP endpoint outage and recovery	Evaluation of queue retention and delayed delivery

The implementation environment reflects the central direction of the project. Edge Sentinel was developed not as a cloud-first surveillance application, but as a local edge prototype that can detect, reason, log, and preserve alerts even when external communication is interrupted.

4.3 Hardware Component Verification

The hardware verification stage confirmed that the main physical components required for the prototype were able to operate with the system. This stage did not attempt to perform long-term reliability testing. Its purpose was to confirm that the camera, PIR sensor, and GSM modem could function as part of the Edge Sentinel hardware design.

The IMX219 NoIR CSI camera was verified for image capture. This confirmed that the camera could supply visual input to the system. The use of a NoIR camera is relevant because the project is motivated partly by surveillance needs in low-light environments. However, camera capture verification does not prove complete night surveillance performance. Low-light and pitch-dark operation still depend on illumination, scene conditions, and further controlled testing.

The HC-SR501 PIR sensor was verified on GPIO17. The verification confirmed that the sensor could produce a motion transition detectable by the Raspberry Pi. This supports the deployment-aware activation direction of the system. The PIR verification should be understood as a functional test, not as a repeated reliability study or a formal latency experiment.

The SIMCOM A7670G modem was verified for SMS delivery through /dev/ttyUSB2 at 115200 baud. This confirmed that the system could communicate with the modem and send an SMS message through the selected serial interface. This result supports the fallback communication direction of the project, especially for environments where internet-based alerting may not always be available. However, the verification does not prove guaranteed SMS delivery, delivery latency, or reliability under all mobile network conditions.

Table 4.2: Hardware Verification Summary

Hardware Component	Verification Performed	Result	Interpretation
IMX219 NoIR CSI camera	Camera capture test	Capture verified	The system can receive visual input from the camera
HC-SR501 PIR sensor	GPIO transition test on GPIO17	Motion transition verified	The system can detect PIR state changes for motion-aware operation
SIMCOM A7670G modem	SMS delivery test through /dev/ttyUSB2 at 115200 baud	SMS delivery verified	The system can send SMS through the modem under the tested condition

The hardware verification results show that the prototype was supported by working physical components. The camera provided the visual foundation, the PIR sensor supported motion-aware input, and the GSM modem supported SMS capability. These results do not remove

the need for future reliability testing, but they confirm that the selected hardware components were usable within the implemented prototype.

4.4 Detector-Tracker Screening and Final Selection

The perception pipeline is a critical part of Edge Sentinel because every behavioural indicator depends on reliable person detection and useful temporal continuity. The system must first identify persons in the scene, then maintain enough continuity across frames to support duration, count, transition, and motion-based rules. For this reason, detector-tracker selection was treated as a system design decision rather than a purely model-comparison exercise.

EfficientDet-Lite0 with ByteTrack was selected as the reference detector-tracker pair for the final evaluation. EfficientDet-Lite0 was selected because it fits the lightweight edge deployment direction of the project. It provides a practical balance between detection capability and computational cost. ByteTrack was selected because it provides anonymous multi-object tracking from detection outputs, allowing the system to reason about movement and persistence without facial recognition or identity classification.

This selection aligns with the project's privacy-conscious and resource-aware design. The system does not need to know who a person is. It only needs to know that a detected person is likely the same temporary track across nearby frames. This makes ByteTrack suitable for indicators such as lingering, tailgating, and abnormal motion. The track identifiers are used only within the system for event reasoning and do not represent personal identity.

The screening process did not produce formal detector mAP or formal tracker metrics such as MOTA, IDF1, or HOTA. This is an important boundary. The project evaluated the perception pipeline mainly by its usefulness to the implemented surveillance rules, its timing behaviour, and its stability under the evaluated workload. The final results should therefore be discussed as system-level and scenario-level evidence, not as a benchmark claim that one detector or tracker is universally superior.

YOLOv8n with ByteTrack was also tested, but only as a sensitivity experiment for the difficult running-past case. That experiment showed stronger instantaneous speed evidence, but it still failed the 0.6-second sustained-duration requirement used by the abnormal-motion rule. It also operated with substantially higher detector latency and lower pipeline throughput compared with the selected reference pipeline. Therefore, YOLOv8n did not replace EfficientDet-Lite0, and no additional YOLOv5n follow-up was required for the final evaluation path.

Table 4.3: Detector-Tracker Selection Summary

Item	Observation	Final Interpretation
Selected reference detector	EfficientDet-Lite0	Used as the final detector for the evaluated reference pipeline
Selected reference tracker	ByteTrack	Used for anonymous multi-object tracking and rule continuity
Main reason for tracking	Behavioural rules require continuity across frames	Tracking supports lingering, tailgating, and abnormal-motion reasoning
Identity handling	Temporary track IDs only	The system does not perform facial recognition or identity classification
YOLOv8n role	Sensitivity analysis for the running-past case	Did not replace EfficientDet-Lite0
YOLOv8n outcome	Stronger instantaneous speed evidence, but still failed sustained-duration rule	The running-past limitation remained unresolved by switching detector
Unsupported metrics	Detector mAP, detector accuracy, MOTA, IDF1, HOTA	These metrics were not claimed because they were not formally evaluated

The detector-tracker screening supports the final architecture of Edge Sentinel. EfficientDet-Lite0 and ByteTrack provided the perception foundation for the evaluated prototype, while the YOLOv8n sensitivity test helped clarify that the running-past false negative was not simply solved by changing detectors. This result strengthened the final claim boundary of the project because it showed where the current rule design works and where it still needs improvement.

4.5 Behavioural Scenario Evaluation

The behavioural scenario evaluation tested whether the implemented indicators produced the expected event outcomes under selected surveillance situations. This evaluation was important because the system was not designed only to detect persons in individual frames. It

was designed to use detections, anonymous tracks, zones, thresholds, durations, transitions, and movement evidence to decide whether a security event had occurred.

A total of nine behavioural scenarios were evaluated. These scenarios covered the implemented indicator families: restricted presence, lingering, crowd density, tailgating, and abnormal motion. Each scenario had an expected outcome, and the system output was compared with that expected outcome. The result was interpreted at the scenario-event level. This means the reported precision, recall, and F1 score describe whether the expected behavioural event outcome was produced, not whether the detector achieved a particular mAP or recognition accuracy.

Out of the nine scenarios, eight produced the correct outcome. One scenario produced a false negative. The false negative occurred in the abnormal-motion scenario identified as `abnormal_motion__running_past_frame`. No false-positive scenarios were recorded. This produced a scenario-level precision of 1.000, a scenario-level recall of 0.875, and a scenario-level F1 score of approximately 0.933.

The high precision indicates that the system did not raise incorrect behavioural alerts in the evaluated scenarios. This is important for surveillance because frequent false alerts can reduce user trust and make operators ignore future warnings. The recall value shows that the system detected most of the expected positive scenarios, but missed one abnormal-motion case. The F1 score gives a balanced summary of the system's event-level performance across the evaluated scenarios.

Table 4.4: Behavioural Scenario Evaluation Summary

Evaluation Item	Result
Total scenarios evaluated	9
Correct outcomes	8
False-negative scenarios	1
False-positive scenarios	0
Scenario-level precision	1.000
Scenario-level recall	0.875
Scenario-level F1 score	Approximately 0.933
Failed scenario	<code>abnormal_motion__running_past_frame</code>
Interpretation boundary	

Evaluation Item	Result
	detector accuracy

Table 4.5: Scenario-Level Outcome Matrix

Scenario Category	Expected Behaviour	System Outcome	Result
Restricted presence/access	Generate event when a person enters or remains in a restricted monitored zone	Expected event produced	Correct
Lingering	Generate event when a tracked person remains beyond the configured duration	Expected event produced	Correct
Crowd density	Generate event when person count exceeds configured threshold	Expected event produced	Correct
Tailgating	Generate event when transition or close-following behaviour satisfies the configured rule	Expected event produced	Correct
Abnormal motion	Generate event when movement satisfies configured abnormal-motion criteria	Expected event produced in most evaluated cases	Mostly correct
Running-past abnormal motion	Generate event for short fast movement across the scene	Event not generated	False negative

The evaluation results show that the rule-based behavioural design was effective across most of the tested scenarios. The result is also useful because it shows both the strength and boundary of the system. Edge Sentinel performed well for the evaluated rule conditions,

especially where events were sustained long enough for detection, tracking, and rule evaluation. However, the running-past false negative shows that very short motion events can challenge rules that require sustained evidence across time.

This outcome supports the design choice of using explainable behavioural indicators. Because the rules are interpretable, the false negative could be analyzed rather than treated as an unexplained model failure. The system did not simply fail silently. The evaluation made it possible to identify why the event was missed and what type of refinement would be needed in future work.

4.6 Running-Past False Negative Analysis

The only failed scenario in the behavioural evaluation was `abnormal_motion__running_past_frame`. This scenario was intended to test whether the abnormal-motion indicator could detect a person moving quickly across the frame. The expected outcome was an abnormal-motion event. The system did not generate the expected event, which made the scenario a false negative.

The failure was not caused by the complete absence of person detection or tracking. The persons involved were detected and tracked, but the tracks were too brief to satisfy the sustained-duration requirement used by the abnormal-motion rule. The rule required movement evidence to persist for a configured duration of 0.6 seconds. In the running-past scenario, the motion was short and passed through the frame quickly. As a result, the system observed fast movement evidence, but not for long enough to trigger the abnormal-motion event under the configured rule.

This result is important because it exposes a real design trade-off. A shorter duration threshold may allow the system to detect very brief running-past events, but it may also increase the risk of false positives from noisy detections, brief tracker jitter, or sudden box movement. A longer duration threshold makes the rule more conservative, but it may miss short events. The prototype chose not to overfit the rule to one difficult clip because doing so could reduce the reliability of the system in other scenarios.

The false negative therefore does not mean that the entire abnormal-motion indicator is unusable. Rather, it shows that the current rule is better suited to sustained abnormal movement than to very short fast crossings. This distinction should be preserved in the final interpretation of the result. The system can support abnormal-motion reasoning under configured conditions, but the present implementation has a known limitation with short-duration motion events.

Table 4.6: Running-Past False Negative Diagnostic Summary

Diagnostic Item	Observation
Failed scenario	abnormal_motion__running_past_frame
Expected event	Abnormal-motion event
Actual outcome	Event not generated
Detection status	Persons were detected
Tracking status	Tracks were produced, but were too brief
Main reason for failure	Sustained-duration requirement was not satisfied
Relevant rule condition	0.6-second sustained-duration requirement
Interpretation	The current abnormal-motion rule is less effective for very short fast crossings
Design decision	The rule was not over-tuned to force one difficult clip to pass

The running-past failure is a useful finding because it improves the honesty of the evaluation. It shows that the system was tested under a condition that exposed a boundary, and the boundary was reported rather than hidden. This strengthens the credibility of the project because the final claims are based on what the prototype actually demonstrated.

4.7 YOLOv8n Sensitivity Experiment

After the running-past false negative was observed, YOLOv8n with ByteTrack was tested as a sensitivity experiment. The purpose of this experiment was to determine whether using a different detector would change the outcome of the difficult short-crossing abnormal-motion scenario. It was not a renewed model-selection campaign, and it was not intended to replace the selected reference pipeline unless it clearly resolved the issue without unacceptable trade-offs.

The YOLOv8n test produced stronger instantaneous speed evidence than the EfficientDet-Lite0 reference pipeline. This suggests that the detector-tracker combination captured some aspects of the fast movement more strongly. However, the event still did not satisfy the 0.6-second sustained-duration requirement. The abnormal-motion event therefore remained undetected under the configured rule.

This result showed that the limitation was not solved simply by switching the detector. The core problem was related to the duration and rule logic of the abnormal-motion indicator. Even when stronger instantaneous speed evidence was available, the movement did not persist long enough to satisfy the sustained condition. This means that future improvement should focus on rule design for short-duration motion, rather than assuming that a heavier detector alone will fix the issue.

The YOLOv8n sensitivity experiment also revealed a performance trade-off. YOLOv8n operated with substantially higher detector latency and lower pipeline throughput than the EfficientDet-Lite0 reference pipeline. Since Edge Sentinel targets Raspberry Pi-class deployment, this trade-off matters. A detector that gives slightly stronger evidence in one diagnostic case may still be unsuitable if it reduces the overall responsiveness or efficiency of the pipeline.

For these reasons, EfficientDet-Lite0 with ByteTrack remained the selected reference detector-tracker pair. YOLOv8n was retained only as a diagnostic comparison for understanding the running-past edge case. No YOLOv5n follow-up was required because the sensitivity result already showed that the main unresolved issue was the sustained-duration rule rather than simply the detector family.

Table 4.7: YOLOv8n Sensitivity Experiment Summary

Item	Observation	Interpretation
Purpose of test	Diagnostic sensitivity analysis for running-past false negative	Not a renewed model-selection campaign
Alternative pipeline	YOLOv8n with ByteTrack	Tested only for the difficult short-crossing case
Observed benefit	Stronger instantaneous speed evidence	YOLOv8n captured some fast movement evidence more strongly
Remaining issue	0.6-second sustained-duration requirement was still not satisfied	The abnormal-motion event still failed
Performance trade-off	Higher detector latency and lower pipeline throughput	Less suitable for the final Raspberry Pi reference pipeline

Item	Observation	Interpretation
Final decision	EfficientDet-Lite0 with ByteTrack remained the reference pair	YOLOv8n did not replace the selected pipeline
Further YOLOv5n follow-up	Not required for final evaluation path	The main limitation was rule duration, not simply detector choice

The YOLOv8n sensitivity experiment helped clarify the nature of the failure. It showed that the running-past scenario was not missed merely because EfficientDet-Lite0 was the selected detector. The missed event was tied to the abnormal-motion rule's requirement for sustained evidence. This finding supports a careful final conclusion: Edge Sentinel demonstrated strong scenario-level performance under the evaluated conditions, but short-duration motion remains a limitation for future refinement.

4.8 Sustained Raspberry Pi Resource Profiling

The sustained resource profiling test was carried out to examine how the selected Edge Sentinel pipeline behaved on the Raspberry Pi during a controlled saved-video workload. This test was important because the project targets constrained edge deployment. A system may produce correct event outcomes in short tests, but still be unsuitable if it overheats, drops frames, suffers unstable processing, or cannot sustain useful throughput on the edge device.

The profiling test used the selected reference pipeline, EfficientDet-Lite0 with ByteTrack. The model was loaded once and warm-up behaviour was excluded from the measured period. The profiling run then measured sustained processing behaviour over the selected workload duration. The main metrics included processed frames per second, detector calls per second, detector latency, end-to-end processing latency, device temperature, throttling status, undervoltage status, dropped or failed frames, and shutdown behaviour.

The profiling result showed approximately 22.72 processed frames per second under the evaluated saved-video workload. The detector was called approximately 11.36 times per second. The mean detector latency was approximately 47.06 ms, while the mean end-to-end processing latency was approximately 43.00 ms as reported by the profiling output. The peak temperature recorded during the run was approximately 49.05°C.

No throttling was recorded during the profiling run. No undervoltage was recorded. No dropped or failed frames were reported. The workload completed with a clean shutdown. These results suggest that the selected reference pipeline was stable under the evaluated saved-video profiling condition on the Raspberry Pi.

The results must be interpreted carefully. The profiling workload was based on saved-video processing, not full live-camera throughput. Therefore, the reported processed FPS should not be presented as guaranteed live-camera FPS in every deployment condition. The test also did not measure electrical power consumption in watts. It can support claims about processing behaviour, latency, temperature, throttling, undervoltage status, and frame processing stability under the tested workload, but it cannot support claims about measured energy consumption or solar endurance.

Table 4.8: Sustained Raspberry Pi Resource Profiling Results

Metric	Result	Interpretation
Reference pipeline	EfficientDet-Lite0 with ByteTrack	Final evaluated detector-tracker pair
Workload type	Saved-video workload	Repeatable profiling condition, not live-camera FPS proof
Processed FPS	Approximately 22.72 FPS	Indicates sustained processing rate under the evaluated workload
Detector calls per second	Approximately 11.36 calls/s	Shows detector scheduling frequency during profiling
Mean detector latency	Approximately 47.06 ms	Indicates average detector inference time
Mean end-to-end processing latency	Approximately 43.00 ms	Indicates average reported pipeline processing latency
Peak temperature	Approximately 49.05°C	Device remained within a safe thermal range during the test
Throttling	None recorded	No thermal or performance throttling observed
Undervoltage	None recorded	No undervoltage condition observed
Dropped or failed frames	None recorded	No frame failure reported during the run
Shutdown behaviour	Clean shutdown	Runtime ended without

Metric	Result	Interpretation
		reported shutdown error

The sustained profiling result supports the claim that the selected pipeline can run stably on the Raspberry Pi under the evaluated workload. This is important for the project because Edge Sentinel is not intended to be only a desktop demonstration. The system is designed for edge deployment, so the resource behaviour of the prototype is part of the evidence for its suitability.

At the same time, the result does not prove every deployment condition. Live camera operation, different scene complexity, stronger low-light processing, larger models, poor power supply, or simultaneous extra services may change runtime behaviour. The result should therefore be used as evidence of stable saved-video profiling on the Raspberry Pi, not as a universal performance guarantee.

4.9 Alert Queue and Recovery Evaluation

The alert queue and recovery evaluation tested one of the central resilience features of Edge Sentinel. Since the project targets infrastructure-constrained environments, it is not enough for the system to generate alerts only when a remote endpoint is immediately available. A practical surveillance system should preserve alerts when delivery fails and deliver them later when communication recovers.

The evaluation simulated a remote HTTP endpoint outage. During the outage, the system continued to generate alerts locally. Since the endpoint was unavailable, the alerts could not be delivered immediately. Instead of losing them, the alert manager stored them in the persistent local queue. After the endpoint was restored, the system retried the queued alerts and delivered them successfully.

A total of ten alerts were generated during the simulated outage. All ten were retained in the queue. After recovery, all ten queued alerts were delivered. No alerts were lost. No duplicate alerts were recorded. The final queue depth was zero. This means that, under the tested condition, the system achieved complete alert retention and complete recovery delivery.

This result is important because it demonstrates that Edge Sentinel can preserve alert records during a remote delivery failure. The system does not stop local event creation simply because the HTTP endpoint is unavailable. It continues to record events and queue failed alerts. This supports the project's resilience claim, especially for environments where remote connectivity may be intermittent.

The evaluation also has important boundaries. The receiver used in the test was local, so the recovery timing represents local queue-flushing overhead rather than internet, cloud, WAN, or SMS latency. The test simulated an HTTP endpoint outage. It was not a full physical network-disconnection test. SMS capability was functionally verified separately, but SMS was not the focus of this queue recovery experiment. Therefore, the result supports HTTP alert queue retention and recovery under the tested simulated outage, not guaranteed alert delivery under every communication failure condition.

Table 4.9: Alert Queue Recovery Results

Evaluation Item	Result	Interpretation
Failure condition	Simulated HTTP endpoint outage	Remote HTTP delivery was made unavailable
Alerts generated during outage	10	Local alert creation continued
Alerts queued	10	Failed alerts were retained locally
Queue retention rate	100 percent	No generated alert was discarded during outage
Alerts delivered after recovery	10	All queued alerts were delivered after endpoint recovery
Recovery delivery rate	100 percent	Queue recovery succeeded under the tested condition
Lost alerts	0	No alert loss observed
Duplicate alerts	0	No duplicate delivery observed
Final queue depth	0	Queue was emptied after successful recovery
Important boundary	Receiver was local	Recovery timing does not represent internet, cloud, WAN, or SMS latency

The alert recovery result is one of the strongest pieces of evidence for the system’s resilience-oriented design. It shows that the system can separate local event creation from immediate remote delivery. This separation matters because infrastructure-constrained environments

may not always provide stable connectivity. By retaining failed alerts and retrying them after recovery, Edge Sentinel avoids one of the major weaknesses of simple alert systems, which is the permanent loss of alerts during delivery failure.

4.10 Local Interface and Configuration Demonstration

The local interface was implemented to support setup, configuration, and review of the Edge Sentinel prototype. This part of the system is important because a surveillance device should not require code-level changes for every deployment environment. Users need a practical way to configure monitoring zones, apply indicator settings, inspect system state, and review generated event evidence.

The interface supports local setup and calibration activities. A camera frame can be used as the basis for zone configuration, allowing monitored regions to be defined according to the actual camera view. This is necessary because every deployment scene is different. A restricted area, doorway, corridor, or monitored boundary must be defined based on the physical environment being observed.

The interface also supports indicator configuration. Behavioural rules depend on settings such as zone assignment, duration threshold, count threshold, and movement conditions. Providing these settings through a local interface makes the system more adaptable. It allows the same prototype to be configured for different monitoring needs without changing the underlying detection and tracking pipeline.

Event and evidence review were also included within the prototype scope. This allows generated events and related evidence to be inspected locally. This feature supports both usability and evaluation. During evaluation, it helps confirm whether events were generated and whether evidence was stored as expected. During deployment-oriented use, it helps users review what happened after an alert or incident.

The interface also provides visibility into alert queue behaviour. This is useful because the resilience layer depends on whether alerts are delivered, queued, retried, or recovered. A user-facing view of queue status makes the alerting system easier to understand and verify. Without such visibility, it would be harder to know whether a failed alert was lost or simply waiting for recovery.

The local interface was treated as an implementation demonstration rather than a formal usability study. The project did not measure user experience, task completion time, or interface satisfaction. However, the existence of the interface strengthens the prototype

because it shows that Edge Sentinel is not only a backend pipeline. It includes practical setup and review mechanisms needed for a configurable surveillance system.

Table 4.10: Local Interface and Configuration Features

Feature	Purpose	Evaluation Interpretation
Local setup interface	Provides access to device setup and configuration functions	Implemented as part of prototype demonstration
Calibration frame support	Allows the camera view to be used for zone configuration	Supports scene-specific setup
Zone configuration	Defines monitored or restricted areas in the camera frame	Supports rule-based event reasoning
Indicator settings	Allows rules and thresholds to be configured	Supports adaptable surveillance behaviour
Event review	Allows generated events to be inspected	Supports verification and system review
Evidence access	Allows event-related evidence to be viewed where available	Supports incident review
Alert queue visibility	Shows queue state and delivery status information	Supports resilience monitoring
Local operation	Allows setup and review near the edge device	Supports edge-first deployment direction

The local interface contributes to the practical value of Edge Sentinel. A surveillance system that is technically functional but difficult to configure may not be useful in real environments. By including local setup, zone configuration, indicator settings, event review, and queue visibility, the prototype moves closer to a deployable edge surveillance tool.

The interface also supports the modular nature of the system. Zones, thresholds, indicators, events, and alerts are not hidden inside one fixed script. They are exposed through configuration and review mechanisms. This makes the prototype easier to demonstrate, evaluate, and extend in future work.

4.11 Discussion Against Objectives

The results of the implementation and evaluation can be discussed in relation to the four objectives stated in Chapter One. This discussion is important because it connects the technical work back to the research aim and shows the extent to which the project was achieved.

The first objective was to design an edge-based surveillance architecture that performs local person detection, anonymous tracking, and rule-based behavioural-event reasoning without requiring continuous cloud connectivity. This objective was achieved. The implemented architecture runs the core perception and reasoning pipeline locally on the Raspberry Pi. Person detection is performed using EfficientDet-Lite0, while ByteTrack provides anonymous track continuity. The behavioural indicators are evaluated locally using detections, tracks, zones, thresholds, and timing information. The system does not require cloud inference before it can generate local events.

The second objective was to implement configurable and explainable rule-based indicators for restricted presence, lingering, crowd density, tailgating, and abnormal motion. This objective was also achieved within the evaluated scenario scope. The prototype includes rule-based indicators that generate events from visible and interpretable conditions. The scenario evaluation showed eight correct outcomes out of nine evaluated scenarios. The only failed case was the short running-past abnormal-motion scenario. This means the indicator layer worked for most tested cases, but the abnormal-motion rule requires refinement for very brief high-speed movement.

The third objective was to develop a local event-logging and alert-delivery mechanism that persistently queues failed HTTP alerts, retries delivery after endpoint recovery, and supports GSM/SMS notification through an AT-command modem. This objective was achieved with clear boundaries. The HTTP alert queue and recovery mechanism was evaluated under a simulated endpoint outage. Ten alerts were generated and queued during the outage, and all ten were delivered after recovery with no losses or duplicates. SMS delivery was also functionally verified through the SIMCOM A7670G modem. However, SMS was not subjected to formal reliability or latency evaluation, and the HTTP recovery experiment did not represent a full physical network-disconnection test.

The fourth objective was to evaluate the prototype using detector-tracker pair screening, labelled behavioural-event scenario outcomes, sustained Raspberry Pi resource profiling, and alert retention and recovery during a simulated HTTP endpoint outage. This objective was achieved. The final reference detector-tracker pair was EfficientDet-Lite0 with ByteTrack. The behavioural evaluation produced scenario-level precision of 1.000, recall of 0.875, and

F1 of approximately 0.933. The sustained Raspberry Pi profiling showed stable saved-video processing with no throttling, no undervoltage, no dropped or failed frames, and clean shutdown. The alert recovery evaluation showed full retention and delivery of queued alerts under the tested endpoint outage condition.

Table 4.11: Objective Achievement Summary

Objective	Achievement Status	Evidence	Limitation
Design an edge-based surveillance architecture for local detection, anonymous tracking, and rule-based reasoning without continuous cloud connectivity	Achieved	Local perception, ByteTrack tracking, rule-based indicators, local logging, and queueing were implemented	Full cloud independence under every deployment condition was not tested
Implement configurable and explainable indicators for restricted presence, lingering, crowd density, tailgating, and abnormal motion	Achieved within scenario scope	Five indicator families were implemented and evaluated across nine behavioural scenarios	One abnormal-motion short-crossing scenario produced a false negative
Develop local logging, HTTP alert queueing, recovery, and GSM/SMS notification capability	Achieved with bounded communication claims	HTTP queue recovery delivered 10 of 10 queued alerts after endpoint recovery; SMS delivery was functionally verified	SMS reliability and latency were not formally evaluated; physical network-disconnection testing was not performed
Evaluate the prototype through detector-tracker screening, scenario	Achieved	EfficientDet-Lite0 + ByteTrack selected; scenario metrics, Pi profiling, and alert	Detector mAP, tracking benchmark metrics, measured power consumption,

Objective	Achievement Status	Evidence	Limitation
evaluation, profiling, and alert recovery testing		recovery results were produced	and live-camera FPS were not evaluated

Overall, the objectives were achieved within the scope of the implemented prototype and the available evaluation evidence. The project demonstrated that a Raspberry Pi-class edge device can support local person detection, anonymous tracking, rule-based behavioural-event reasoning, local event preservation, and alert recovery under a simulated endpoint outage. The results also identified areas that should be improved before the system can be considered field-ready.

4.12 Limitations and Threats to Validity

Although the evaluation results are promising, the findings must be interpreted within the limits of the implemented prototype and the tests performed. The project provides evidence for a working resilience-oriented edge surveillance prototype, but it does not prove every possible deployment condition.

The first limitation is that the project did not report formal detector accuracy metrics. Metrics such as mAP, detector precision, detector recall, or detector F1 were not measured using a ground-truth annotated detection dataset. The reported precision, recall, and F1 score are scenario-level behavioural outcome metrics. They describe whether the expected event occurred in each evaluated scenario, not how accurately the detector localized every person in every frame.

The second limitation is that formal tracking benchmark metrics were not evaluated. The system uses ByteTrack for anonymous track continuity, but metrics such as MOTA, IDF1, and HOTA were not measured. The tracker was evaluated indirectly through its usefulness to the rule-based indicators. This is acceptable for the project scope, but it means the work should not claim formal tracking benchmark performance.

The third limitation is the abnormal-motion false negative. The scenario `abnormal_motion__running_past_frame` did not trigger the expected event because the observed motion did not satisfy the sustained-duration requirement. This shows that the present abnormal-motion logic is better suited to sustained movement than to very short fast crossings. Future work should consider improved short-duration motion handling, possibly by adjusting temporal rules, using adaptive thresholds, or adding a separate short-burst motion condition.

The fourth limitation concerns low-light evaluation. The project was motivated partly by the need for surveillance under poor lighting, and the IMX219 NoIR camera was verified for capture. However, a controlled low-light or pitch-dark evaluation campaign was not completed. Therefore, the project should not claim proven night surveillance or pitch-dark performance. Low-light testing with adequate infrared illumination remains a future evaluation need.

The fifth limitation concerns power evaluation. The project is motivated by unreliable power conditions and includes power-conscious design ideas such as PIR-assisted activation. However, electrical power consumption in watts was not measured. Solar or battery autonomy was also not formally tested. The sustained profiling result showed no throttling, no undervoltage, and acceptable thermal behaviour under the evaluated workload, but this should not be interpreted as a complete power study.

The sixth limitation concerns network resilience. The alert queue recovery experiment simulated an HTTP endpoint outage and showed that alerts were retained and delivered after recovery. However, the receiver was local, and the experiment did not test full physical network disconnection, WAN failure, mobile network failure, or cloud deployment. The result supports queue retention and recovery under the tested condition, but not guaranteed delivery under all network failures.

The seventh limitation concerns SMS communication. SMS delivery was functionally verified using the SIMCOM A7670G modem, but repeated SMS reliability, delivery latency, and failure recovery were not formally evaluated. Therefore, SMS should be described as a verified capability, not as a guaranteed communication channel.

The eighth limitation concerns live-camera performance. The sustained profiling test used a saved-video workload. This allowed repeatable measurement and stable comparison, but it should not be described as live-camera FPS. Live-camera performance may be affected by camera capture settings, lighting, concurrent services, storage speed, UI usage, and other runtime conditions.

The ninth limitation concerns deployment scale. The project implemented a local prototype, not a full multi-device cloud platform. Features such as remote account management, multi-site dashboards, continuous cloud synchronization, and large-scale deployment management were outside the completed evaluation scope. These may be important for future commercialization, but they are not completed thesis achievements.

Table 4.12: Limitations and Threats to Validity

Limitation Area	Description	Effect on Interpretation
Detector metrics	mAP and detector-level accuracy metrics were not measured	Scenario metrics must not be described as detector accuracy
Tracking metrics	MOTA, IDF1, and HOTA were not measured	Tracking claims should be limited to anonymous continuity for rule reasoning
Abnormal motion	One short running-past case produced a false negative	The abnormal-motion rule needs refinement for brief fast crossings
Low-light testing	Controlled low-light or pitch-dark evaluation was not completed	No claim of proven night or pitch-dark surveillance performance
Power measurement	Watt-level consumption and solar autonomy were not measured	Resource profiling should not be interpreted as power consumption evaluation
Network resilience	HTTP endpoint outage was simulated with a local receiver	The result does not prove all physical network, cloud, or WAN failure conditions
SMS evaluation	SMS delivery was functionally verified but not repeatedly evaluated	SMS claims should be bounded to functional verification
Live-camera throughput	Profiling used saved-video workload	Reported FPS should not be treated as universal live-camera FPS
Deployment scale	Multi-device cloud dashboard and account management were not completed	The prototype should be described as local and edge-based

These limitations do not invalidate the project. Instead, they define the boundary of what the current evidence supports. A strong final-year project does not need to claim that every possible problem has been solved. It needs to show that the implemented system addresses

the stated aim, that the evaluation was honest, and that the remaining limitations are clearly understood.

4.13 Summary of Findings

This chapter presented the implementation, results, and discussion of the Edge Sentinel prototype. The results show that the system was successfully implemented as a modular edge-based surveillance prototype using Raspberry Pi-class hardware. The implemented system includes local person detection, anonymous tracking, configurable behavioural indicators, event logging, evidence handling, persistent alert queueing, HTTP alert recovery, local configuration, and GSM/SMS capability.

The hardware verification confirmed that the IMX219 NoIR camera could capture visual input, the HC-SR501 PIR sensor could produce a detectable transition on GPIO17, and the SIMCOM A7670G modem could send SMS through the selected serial interface. These results confirmed the functional readiness of the main hardware components used in the prototype.

The detector-tracker screening led to the selection of EfficientDet-Lite0 with ByteTrack as the reference perception pipeline. This pair supported the project's edge-focused design by combining lightweight person detection with anonymous track continuity. YOLOv8n with ByteTrack was tested only as a sensitivity experiment for the running-past false negative and did not replace the selected reference pipeline.

The behavioural scenario evaluation produced eight correct outcomes out of nine scenarios. There were no false-positive scenarios. The scenario-level precision was 1.000, recall was 0.875, and F1 score was approximately 0.933. The only false negative occurred in the abnormal-motion running-past case, where the track evidence did not satisfy the 0.6-second sustained-duration requirement. This result showed that the current abnormal-motion rule is limited for short fast crossings.

The sustained Raspberry Pi profiling result showed approximately 22.72 processed FPS and approximately 11.36 detector calls per second under the evaluated saved-video workload. The mean detector latency was approximately 47.06 ms, and the peak temperature was approximately 49.05°C. No throttling, undervoltage, dropped frames, failed frames, or shutdown errors were recorded. These results support the stability of the selected reference pipeline under the tested workload.

The alert queue recovery evaluation provided strong evidence for the resilience direction of the project. During a simulated HTTP endpoint outage, ten alerts were generated and queued.

After endpoint recovery, all ten alerts were delivered, with no lost alerts, no duplicates, and final queue depth of zero. This result shows that Edge Sentinel can preserve failed alerts locally and recover delivery when the endpoint becomes available again.

Overall, the findings show that Edge Sentinel achieved its main aim within the implemented scope. The prototype demonstrates that local person detection, anonymous tracking, explainable rule-based event reasoning, and persistent alert recovery can be combined on an edge device for infrastructure-constrained surveillance. The project does not claim perfect anomaly detection, formal detector accuracy, measured power consumption, full network-failure validation, or guaranteed SMS reliability. Instead, it provides a bounded and defensible demonstration of a resilience-oriented edge surveillance architecture.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter presents the conclusion and recommendations of the project. The chapter summarizes the work carried out, explains the major findings, states the contribution of the project, identifies the limitations of the completed prototype, and presents recommendations for future improvement.

The aim of this project was to develop and evaluate a resilient edge-based intelligent visual surveillance prototype that performs local person detection, anonymous tracking, rule-based behavioural-event reasoning, and persistent alert handling for infrastructure-constrained environments. The work was motivated by the limitations of passive CCTV monitoring, cloud-dependent surveillance systems, unreliable connectivity, unstable power supply, and the need for practical security systems that can operate close to where events occur.

Edge Sentinel was developed as a modular prototype that combines local computer vision, anonymous multi-object tracking, configurable behavioural indicators, local event logging, event evidence, alert queueing, HTTP recovery, and GSM/SMS capability. The project did not attempt to build a facial recognition system or a universal anomaly detection model. Instead, it focused on explainable surveillance events and resilience-oriented operation on Raspberry Pi-class edge hardware.

5.2 Summary of the Work Carried Out

The project began with the identification of a practical surveillance problem. Traditional CCTV systems are useful for recording and reviewing incidents, but they often remain passive because they depend on human operators to monitor footage or review recordings after an incident. This limitation becomes more serious in environments where security staff, stable power supply, reliable internet, and continuous monitoring cannot be guaranteed.

A literature review was carried out across intelligent visual surveillance, edge computing, low-light computer vision, object detection, multi-object tracking, video anomaly detection, edge AI optimization, and infrastructure-constrained security systems. The review showed that many existing works focus on model accuracy, anomaly detection, or low-light enhancement, while fewer works address the complete system problem of local inference,

tracking, explainable event reasoning, alert retention, and fallback communication in constrained environments.

Based on the identified gap, Edge Sentinel was designed using an iterative prototype-based development methodology. This approach allowed the system to evolve through repeated cycles of design, implementation, testing, evaluation, and refinement. The final prototype was structured around sensing, perception, configuration, behavioural reasoning, event and evidence handling, alert communication, resilience, and local interface layers.

The implemented system used Raspberry Pi 5 as the edge platform. The IMX219 NoIR CSI camera provided visual input, the HC-SR501 PIR sensor supported motion-aware input, and the SIMCOM A7670G modem provided GSM/SMS capability. EfficientDet-Lite0 was selected as the reference detector, while ByteTrack was used for anonymous multi-object tracking. The system used rule-based behavioural indicators for restricted presence, lingering, crowd density, tailgating, and abnormal motion.

The prototype was evaluated using hardware verification, detector-tracker screening, behavioural scenario evaluation, sustained Raspberry Pi profiling, and alert queue recovery testing. The hardware verification confirmed that the camera could capture images, the PIR sensor could produce a detectable transition on GPIO17, and the modem could send SMS through the selected serial interface. The behavioural evaluation tested nine scenarios and produced eight correct outcomes, one false negative, and no false-positive scenarios. The sustained profiling test showed stable saved-video processing on the Raspberry Pi, while the alert recovery test showed that failed HTTP alerts could be queued and delivered after endpoint recovery.

5.3 Conclusion

The project successfully developed and evaluated Edge Sentinel as a resilience-oriented edge-based intelligent visual surveillance prototype. The system demonstrated that local person detection, anonymous tracking, configurable rule-based event reasoning, local event preservation, and persistent alert recovery can be combined on Raspberry Pi-class hardware.

The results show that the system achieved its main aim within the implemented scope. EfficientDet-Lite0 with ByteTrack provided the final reference perception pipeline. The behavioural indicators produced correct outcomes in eight out of nine evaluated scenarios, with scenario-level precision of 1.000, recall of 0.875, and F1 score of approximately 0.933. The only failed scenario was the short running-past abnormal-motion case, where the observed track did not satisfy the sustained-duration requirement. This finding identified a specific limitation rather than a general failure of the behavioural reasoning design.

The sustained Raspberry Pi profiling also supported the practical edge direction of the project. Under the evaluated saved-video workload, the system processed approximately 22.72 frames per second, made approximately 11.36 detector calls per second, recorded no throttling, no undervoltage, no dropped or failed frames, and completed with a clean shutdown. These results show that the selected reference pipeline was stable under the tested workload.

The alert queue recovery evaluation provided strong support for the resilience contribution of the project. During a simulated HTTP endpoint outage, ten alerts were generated and queued. After endpoint recovery, all ten alerts were delivered, with no lost alerts, no duplicate alerts, and final queue depth of zero. This shows that Edge Sentinel can preserve alerts locally when remote delivery fails and recover delivery when the endpoint becomes available again.

The project therefore concludes that a practical surveillance system for infrastructure-constrained environments should not depend only on model accuracy or continuous cloud connectivity. It should be designed as a complete edge system with local intelligence, explainable event rules, event preservation, and communication recovery. Edge Sentinel demonstrates this direction through an implemented and evaluated prototype.

5.4 Contribution of the Project

The main contribution of this project is the development of a modular and resilience-oriented edge surveillance prototype for infrastructure-constrained environments. The project contributes a system design that combines local person detection, anonymous tracking, configurable behavioural indicators, event evidence, persistent alert queueing, and fallback communication capability.

The project also contributes an explainable approach to behavioural surveillance. Instead of relying entirely on a heavy black-box anomaly detection model, the system uses interpretable rules based on zones, duration, count, transitions, and motion evidence. This makes the generated events easier to understand, tune, and evaluate. It also makes the system more suitable for constrained edge hardware.

Another contribution is the integration of alert resilience into the surveillance architecture. The system does not discard alerts when HTTP delivery fails. It stores them in a persistent queue and retries them after recovery. This is important for environments where connectivity may be unstable. The alert recovery result shows that resilience can be treated as a measurable part of surveillance system evaluation, not only as a design intention.

The project also contributes a bounded evaluation approach. Rather than claiming unsupported detector accuracy or universal anomaly detection, the system was evaluated using scenario-level event outcomes, edge-device profiling, alert recovery testing, and hardware verification. This makes the findings more honest and easier to defend.

5.5 Limitations of the Project

The completed prototype has some limitations. The project did not measure detector mAP, detector-level precision, detector-level recall, or detector-level F1 score using a fully annotated detection dataset. The reported precision, recall, and F1 score are scenario-level behavioural outcome metrics, not detector benchmark metrics.

The project also did not evaluate formal tracking metrics such as MOTA, IDF1, or HOTA. ByteTrack was used to provide anonymous continuity for event reasoning, and its usefulness was evaluated through the behaviour of the implemented indicators. A separate tracking benchmark was outside the completed scope.

The abnormal-motion rule also has a known limitation. The running-past scenario produced a false negative because the movement did not satisfy the 0.6-second sustained-duration requirement. This shows that the current abnormal-motion indicator is less effective for very brief fast crossings and would benefit from further refinement.

Low-light operation remains another limitation. The project used and verified the IMX219 NoIR camera, but a controlled low-light or pitch-dark evaluation campaign was not completed. The project therefore cannot claim proven night surveillance or pitch-dark performance.

Power evaluation was also limited. The system was motivated by unreliable power conditions and included PIR-assisted activation as a power-conscious direction, but watt-level power consumption, solar autonomy, and battery endurance were not formally measured. The profiling result supports device stability under the evaluated workload, but it does not represent a full power study.

The network-resilience test was also bounded. The alert recovery experiment simulated an HTTP endpoint outage and used a local receiver. It demonstrated queue retention and recovery under the tested condition, but it did not represent all possible physical network failures, cloud failures, WAN delays, or mobile network conditions. SMS delivery was functionally verified, but repeated SMS reliability and latency testing were not performed.

5.6 Recommendations

Future work should improve the abnormal-motion indicator so that short and fast movements can be detected more reliably without increasing false positives. This may require a separate short-burst motion rule, adaptive duration thresholds, improved track smoothing, or a more refined movement scoring method.

A more detailed detector and tracker evaluation should also be carried out. Future versions of the system should evaluate detector-level metrics using annotated frames and tracking metrics using suitable tracking ground truth. This would make it possible to compare detection and tracking performance more formally, beyond scenario-level event outcomes.

Further low-light testing is also recommended. The system should be tested under controlled lighting levels, including daylight, evening, dim indoor lighting, infrared illumination, and near-dark conditions. This would help determine how reliably the detector-tracker pipeline works under poor illumination and whether low-light enhancement is necessary.

Power measurement should be added in future work. A proper power meter or measurement setup should be used to record actual watt-level consumption during idle, active processing, alert delivery, and sensor-triggered operation. This would provide stronger evidence for deployment planning, especially where solar and battery operation are desired.

The alerting system should also be tested under more realistic network conditions. Future tests should include physical network disconnection, unstable Wi-Fi, mobile data failure, delayed endpoint recovery, and SMS delivery under different signal conditions. This would make the resilience evaluation closer to real deployment environments.

The local interface can also be improved. Future work should refine the user experience for zone drawing, indicator assignment, event review, evidence browsing, and alert queue monitoring. A more polished interface would make the system easier to configure and demonstrate in real environments.

Finally, future work may extend Edge Sentinel into a more complete deployment platform. This could include multi-device management, secure remote access, cloud synchronization, role-based access control, encrypted evidence storage, and longer field trials. These additions should build on the current edge-first architecture without removing the system's local processing and alert-retention strengths.

5.7 Summary of Chapter

This chapter concluded the project by summarizing the work carried out, stating the major findings, identifying the contribution of the project, discussing limitations, and presenting recommendations for future improvement. The project achieved its aim by developing and evaluating a resilient edge-based intelligent visual surveillance prototype for infrastructure-constrained environments.

The final system demonstrated local person detection, anonymous tracking, explainable behavioural-event reasoning, local event logging, alert queue recovery, and GSM/SMS capability. The evaluation showed strong scenario-level performance, stable saved-video profiling on the Raspberry Pi, and successful alert retention and recovery under a simulated HTTP endpoint outage.

The project does not claim to solve every surveillance problem. It does not claim perfect anomaly detection, formal detector accuracy, measured power consumption, complete low-light performance, or guaranteed communication reliability. Its contribution is more focused and more practical. Edge Sentinel shows that a modular edge surveillance system can preserve useful local intelligence and alert records even when infrastructure conditions are limited.

REFERENCES

- Abu Awwad, Y., Rana, O., & Perera, C. (2024). Anomaly detection on the edge using smart cameras under low-light conditions. *Sensors*, 24(3), 772. <https://doi.org/10.3390/s24030772>
- Ali, M., Goyal, L., Sharma, C. M., & Kumar, S. (2024). Edge-computing-enabled abnormal activity recognition for visual surveillance. *Electronics*, 13(2), 251. <https://doi.org/10.3390/electronics13020251>
- Aliyu, A.-Q. H., & Airoboman, A. E. (2025). Design and implementation of a cost-effective, AI-enabled smart security system for real-time threat detection in Nigerian rural communities. *Journal of Scientific and Engineering Research*, 12(11), 42–58.
- Batzner, K., Heckler, L., & König, R. (2023). EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 12821–12830.
- Cheng, Y., Wang, D., Zhou, P., & Zhang, T. (2018). A survey of model compression and acceleration for deep neural networks. *IEEE Signal Processing Magazine*, 35(1), 126–136. <https://doi.org/10.1109/MSP.2017.2765695>
- Duong, H.-T., Le, V.-T., & Hoang, V. T. (2023). Deep learning-based anomaly detection in video surveillance: A survey. *Sensors*, 23(11), 5024. <https://doi.org/10.3390/s23115024>
- Elmetwally, A., Eldeeb, R., & Elmougy, S. (2025). Deep learning based anomaly detection in real-time video. *Multimedia Tools and Applications*, 84, 9555–9571. <https://doi.org/10.1007/s11042-024-19116-9>
- Ghari, B., Tourani, A., Shahbahrami, A., & Gaydadjiev, G. (2024). Pedestrian detection in low-light conditions: A comprehensive survey. *Image and Vision Computing*, 148, 105106. <https://doi.org/10.1016/j.imavis.2024.105106>
- Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2021). A survey of quantization methods for efficient neural network inference. *arXiv*. <https://arxiv.org/abs/2103.13630>
- Hashmi, K. A., Kallempudi, G., Stricker, D., & Afzal, M. Z. (2023). FeatEnHancer: Enhancing hierarchical features for object detection and beyond under low-light vision. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 6725–6735. <https://doi.org/10.1109/ICCV51070.2023.00619>

Li, C., Guo, C., Han, L. H., Jiang, J., Cheng, M.-M., Gu, J., & Loy, C. C. (2022). Low-light image and video enhancement using deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(12), 9396–9416. <https://doi.org/10.1109/TPAMI.2021.3126387>

Liu, W., Luo, W., Lian, D., & Gao, S. (2018). Future frame prediction for anomaly detection: A new baseline. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 6536–6545. <https://doi.org/10.1109/CVPR.2018.00684>

Mehran, R., Oyama, A., & Shah, M. (2009). Abnormal crowd behavior detection using social force model. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 935–942. <https://doi.org/10.1109/CVPR.2009.5206641>

Nasir, R., Jalil, Z., Nasir, M., Alsubait, T., Ashraf, M., & Saleem, S. (2025). An enhanced framework for real-time dense crowd abnormal behavior detection using YOLOv8. *Artificial Intelligence Review*, 58, 202. <https://doi.org/10.1007/s10462-025-11206-w>

Nejad, S. S., & Haque, A. (2024). Weakly-supervised anomaly detection in surveillance videos based on two-stream I3D convolution network. *arXiv*. <https://arxiv.org/abs/2411.08755>

Nte, N. D., Gande, G., & Uzorka, M. (2020). The challenges and prospects of ICTs in crime prevention and management in Nigeria: A review of CCTV cameras in Abuja. *Indonesian Journal of Criminal Law Studies*, 5(1), 75–100. <https://doi.org/10.15294/ijcls.v5i1.36372>

Qu, J., Liu, R. W., Gao, Y., Guo, Y., Zhu, F., & Wang, F.-Y. (2024). Double domain guided real-time low-light image enhancement for ultra-high-definition transportation surveillance. *IEEE Transactions on Intelligent Transportation Systems*, 25(8), 9550–9562.

Tan, M., Pang, R., & Le, Q. V. (2020). EfficientDet: Scalable and efficient object detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 10781–10790.

Tran, D. Q., Aboah, A., Jeon, Y., Shoman, M., Park, M., & Park, S. (2024). Low-light image enhancement framework for improved object detection in fisheye lens datasets. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 7056–7065.

Tusher, M. B. U., Akash, S. K., & Showmik, A. I. (2025). Anomaly detection using computer vision: A comparative analysis of class distinction and performance metrics. *arXiv*. <https://arxiv.org/abs/2503.19100>

Wu, J. (2023). Accuracy improvement of edge image recognition AI in dark light environment by image enhancement. *Proceedings of the 3rd International Conference on Signal Processing and Machine Learning*. <https://doi.org/10.54254/2755-2721/4/2023377>

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, & T. Hassner (Eds.), *Computer Vision – ECCV 2022* (pp. 1–21). Springer.

Zhou, Y., Liu, D., & Huang, T. (2018). Survey of face detection on low-quality images. *Proceedings of the 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2018)*, 769–773. <https://doi.org/10.1109/FG.2018.00121>

APPENDICES

Appendix A: Project Source Code and Repository Structure

[To be added.]

Appendix B: System Configuration Samples

[To be added.]

Appendix C: Evaluation Artifacts and Runtime Evidence

[To be added.]

Appendix D: Additional System Diagrams

[To be added.]

Appendix E: Hardware Verification Notes

[To be added.]